Machine Learning L+Pr

Béla J. Szekeres, PhD

Lecture 3

ELTE Faculty of Informatics, Szombathely, Hungary

1. Overview
2. Normalization
3. K-Nearest Neighbor Classifier
   - Distance Metrics
   - Example
4. Cross-Validation
   - Overfitting
   - Underfitting
   - K-Folds Cross Validation
5. Clustering
   - K-Means
   - Finding k
   - DBSCAN
   - Hierarchical clustering
6. PCA

Topics of this day

- Data preparation: normalization
- KNN
- Distance metrics
- Overfitting/Underfitting and Cross-Validation
- Clustering: K-Means, Hierarchical, DBScan
- PCA
- Practice: Clustering

Normalization

- Features are usually transformed into a range before doing KNN. **Why?**
- The distance formula depends on how features are measured
- if certain features have much larger values than others the distance measurements will be strongly **dominated by the larger values**
- we have to rescale them (to bring them to the same scale) $\Rightarrow$ each one should contribute relatively equally
- We'll see later, that normalization is recommended for gradient descent.

Types

- Min-max normalization
- Z-transformation (standardization)

Normalization

Min-max

- this process transforms each feature such that all of its values fall in a range between 0 and 1
- $X_{new} = \frac{X - \min(X)}{max(X) - min(X)}$

Z-score

- this process transforms each feature such that the values are centered around the mean with a unit standard deviation. This means that the mean of the attribute becomes zero and the resultant distribution has a unit standard deviation.
- $X_{new} = \frac{X - \ mean(X)}{std(X)}$ $\quad std(X) = \sqrt{1/N \sum_{i=1}^{N} (X^{(i)} - \overline{X})^2}$

K-Nearest Neighbor Classifier (k-NN)

- it can classify examples by assigning them the class of the **most similar labeled examples**
- very simple but extemely powerful algorithm
- k-NN is well suited for classification tasks where the relationships between the features are very complex and hard to understand

The algorithm

- we have a **training dataset** so example that are classified into several categories
- we have a new example (with the same number of features as the training data), and k-NN identifies k elements in the train dataset that the "nearest" in similarity.
- the unlabeled new example is assigned to the class of the majority of the k nearest neighbors

Distance metrics

How to measure the distance?

- **Euclidean distance**: $d(\vec{x}, \vec{y}) = \sqrt{\sum_{i=1}^{K} (x_i - y_i)^2}$
  - usually not recommended in higher dimensions
  - **scales poorly** with the number of features
- **Manhattan distance**: $d(\vec{x}, \vec{y}) = \sum_{i=1}^{K} |x_i - y_i|$
  - it is usually preferred
  - **scales well** with the number of features

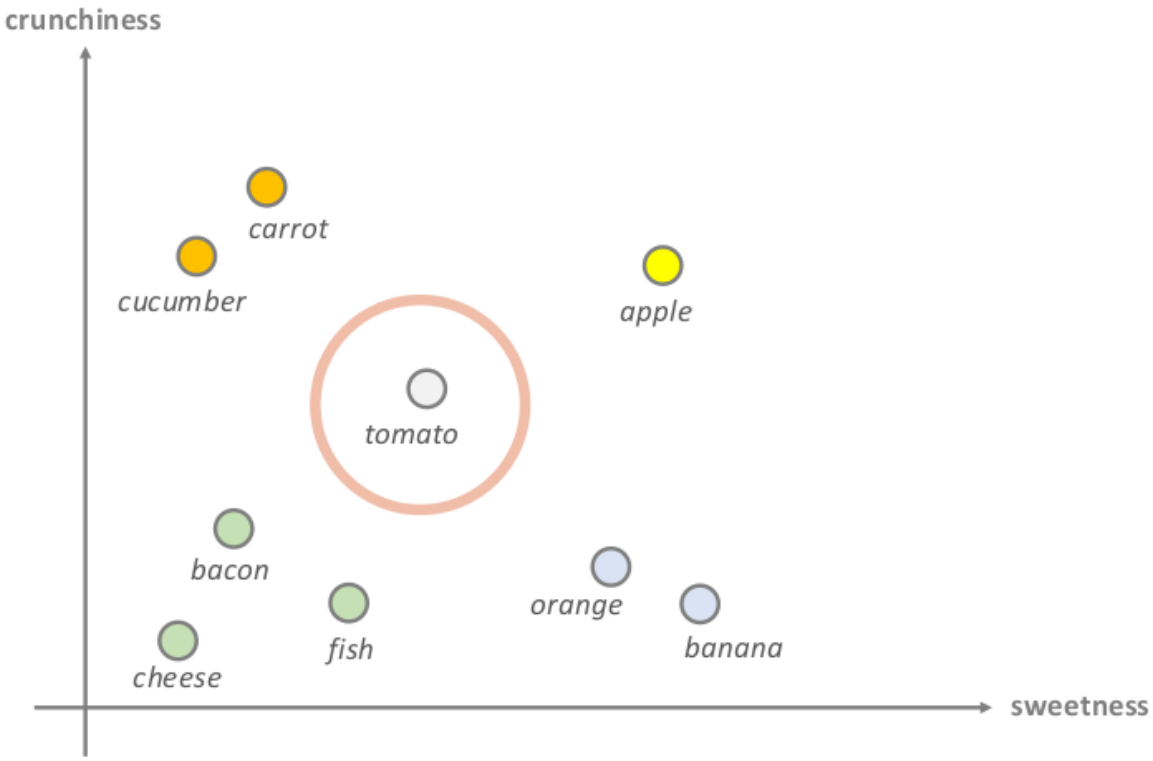
Example

Task

What type of ingredient is **tomato**?

Ingredient	Sweetness	Crunchiness	Type
apple	10	9	fruit
bacon	1	4	protein
banana	10	1	fruit
banana	9	2	fruit
carrot	7	10	vegetable
cucumber	6	9	vegetable
cheese	1	1	protein
fish	2	1	protein
tomato	6	4	???

Example



Problem

For different k we predict another class. Hmmm.

How to choose k? Answer: by CV.

Cross-validation: Motivation

We fit the model on the **training** dataset then we test the model on the **test dataset**. We only have information how the model performs to our in-sample data but we would like to see the accuracy when dealing with new data

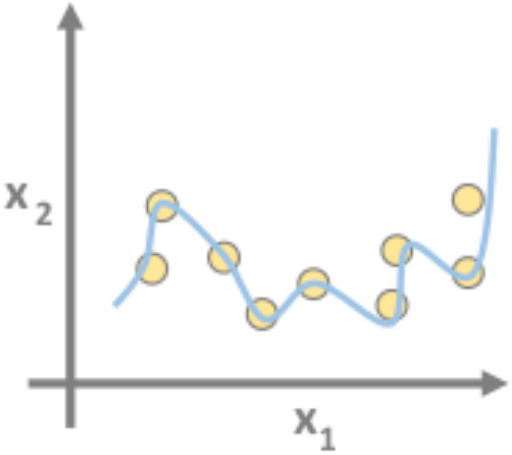
Aim

Choosing that model, which generalizes the best.

Overfitting

What is overfitting?

Model has trained "too well" on the training set.

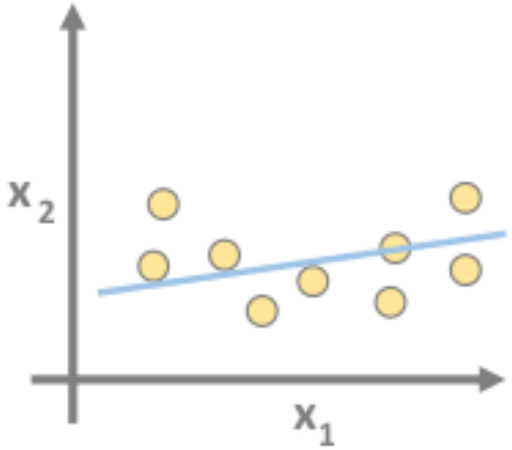


- it means it is very accurate on the training set but yields pooer results on the test set $\rightarrow$ **Poor generalization**
- the model learns the "noise" instead of the actual relationships between the variables

Underfitting

What is underfitting?

Model has not been trained "well" on the training set.



- this is usually the case

when we use too simple models for the problem

Use **K-Folds Cross Validation** — Special type of CV

K-Folds Cross Validation

- The aim is to be able to **generalize the model** to new datasets with same accuracy.
- We use all the data for training the model
- **Split** the data into k folds (for example $k = 5$).

Aim

Choosing that model, which generalizes the best.

K-Folds Cross Validation

- The aim is to be able to **generalize the model** to new datasets with same accuracy.
- We use all the data for training the model
- **Split** the data into k folds (for example $k = 5$).

We run k separate learning experiment with $k - 1$ folds and 1 fold for the test set and finally we average the results from this k pieces of the **test results**.

K-Folds Cross Validation

- The aim is to be able to **generalize the model** to new datasets with same accuracy.
- We use all the data for training the model
- **Split** the data into k folds (for example $k = 5$).

We run k separate learning experiment with $k - 1$ folds and 1 fold for the test set and finally we average the results from this k pieces of the **test results**.

K-Folds Cross Validation

- The aim is to be able to **generalize the model** to new datasets with same accuracy.
- We use all the data for training the model
- **Split** the data into k folds (for example $k = 5$).

We run k separate learning experiment with $k - 1$ folds and 1 fold for the test set and finally we average the results from this k pieces of the **test results**.

K-Folds Cross Validation

- The aim is to be able to **generalize the model** to new datasets with same accuracy.
- We use all the data for training the model
- **Split** the data into k folds (for example $k = 5$).

We run k separate learning experiment with $k - 1$ folds and 1 fold for the test set and finally we average the results from this k pieces of the **test results**.

K-Folds Cross Validation

- The aim is to be able to **generalize the model** to new datasets with same accuracy.
- We use all the data for training the model
- **Split** the data into k folds (for example $k = 5$).

Huge advantage

All observations are used for both training and validation and each observation are used for validation **exactly once**.

Clustering

- clustering is the task of grouping a set of objects in such a way that objects in the same group (called a cluster) are more similar (in some sense) to each other than to those in other groups (clusters).
- **unsupervised learning**

K-Means

- **k-means** clustering aims to partition N observations into k clusters in which each observation belongs to the cluster with the nearest **center**
- it is an **NP-hard** problem

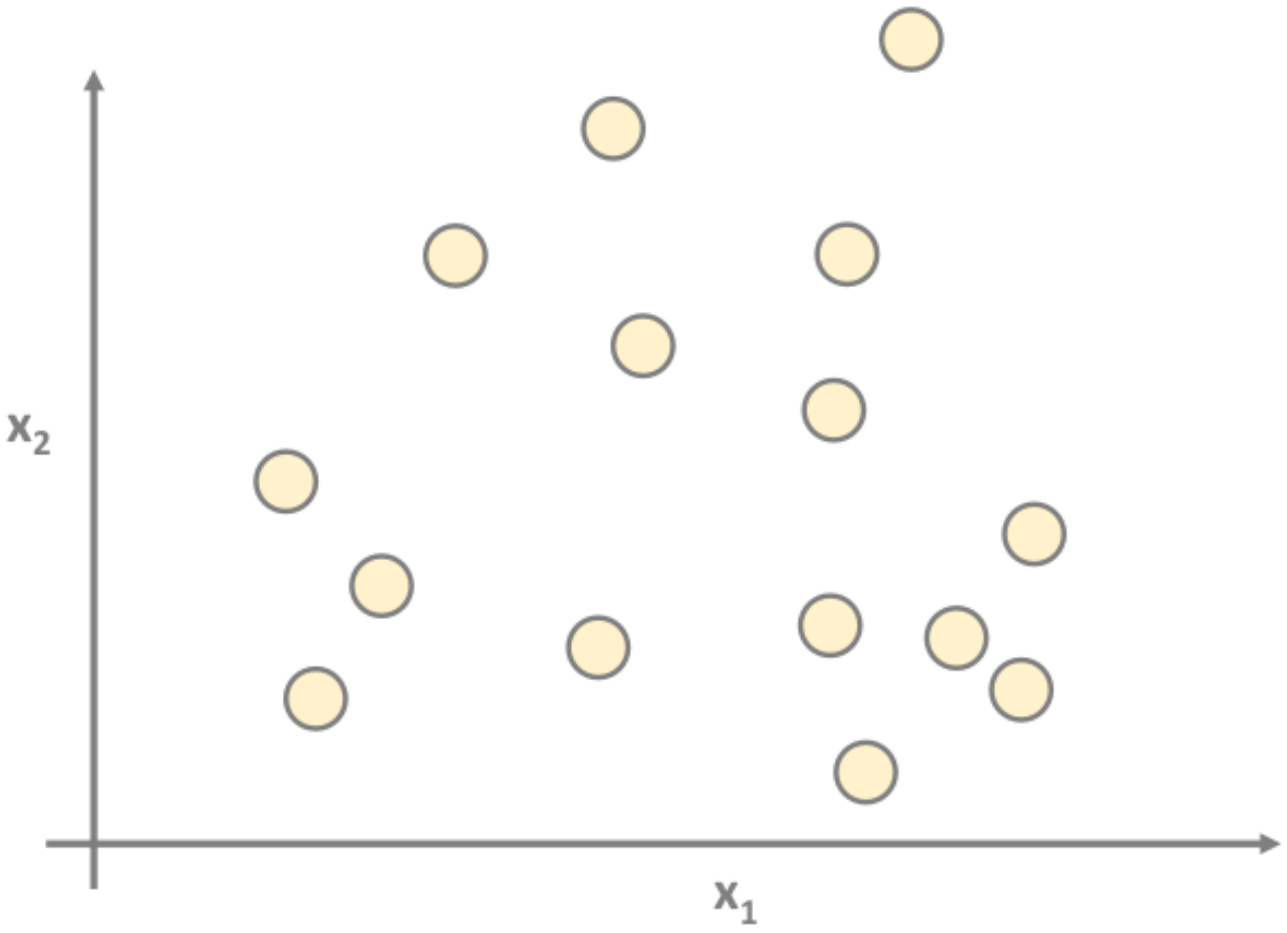
The algorithm

Initialize the centrums (or centroids) at random.

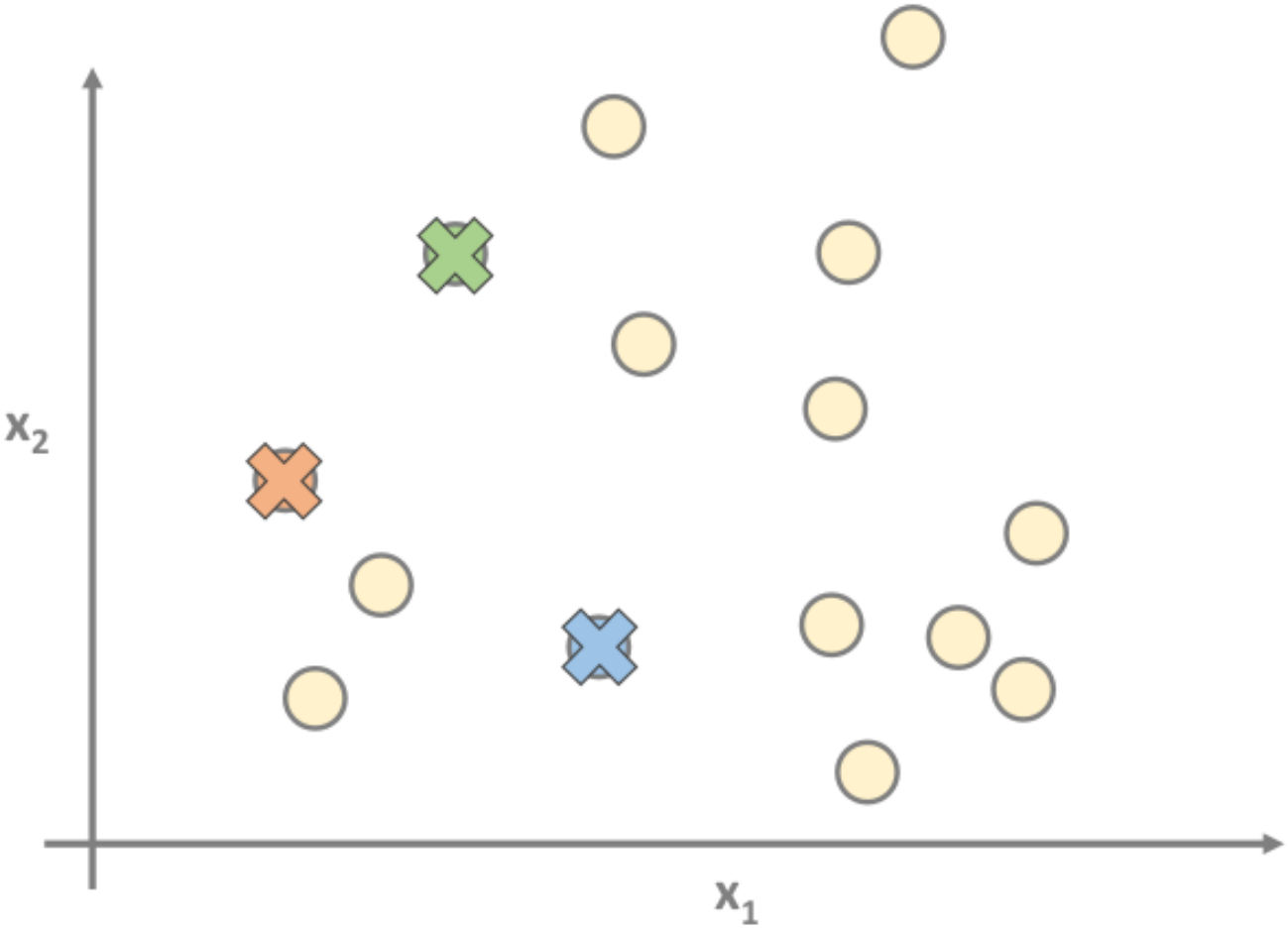
Do until convergence

- decide for every point in our dataset what centroid is the nearest to them
- calculate the **new centers** (it's the mean of them) of every distinct cluster

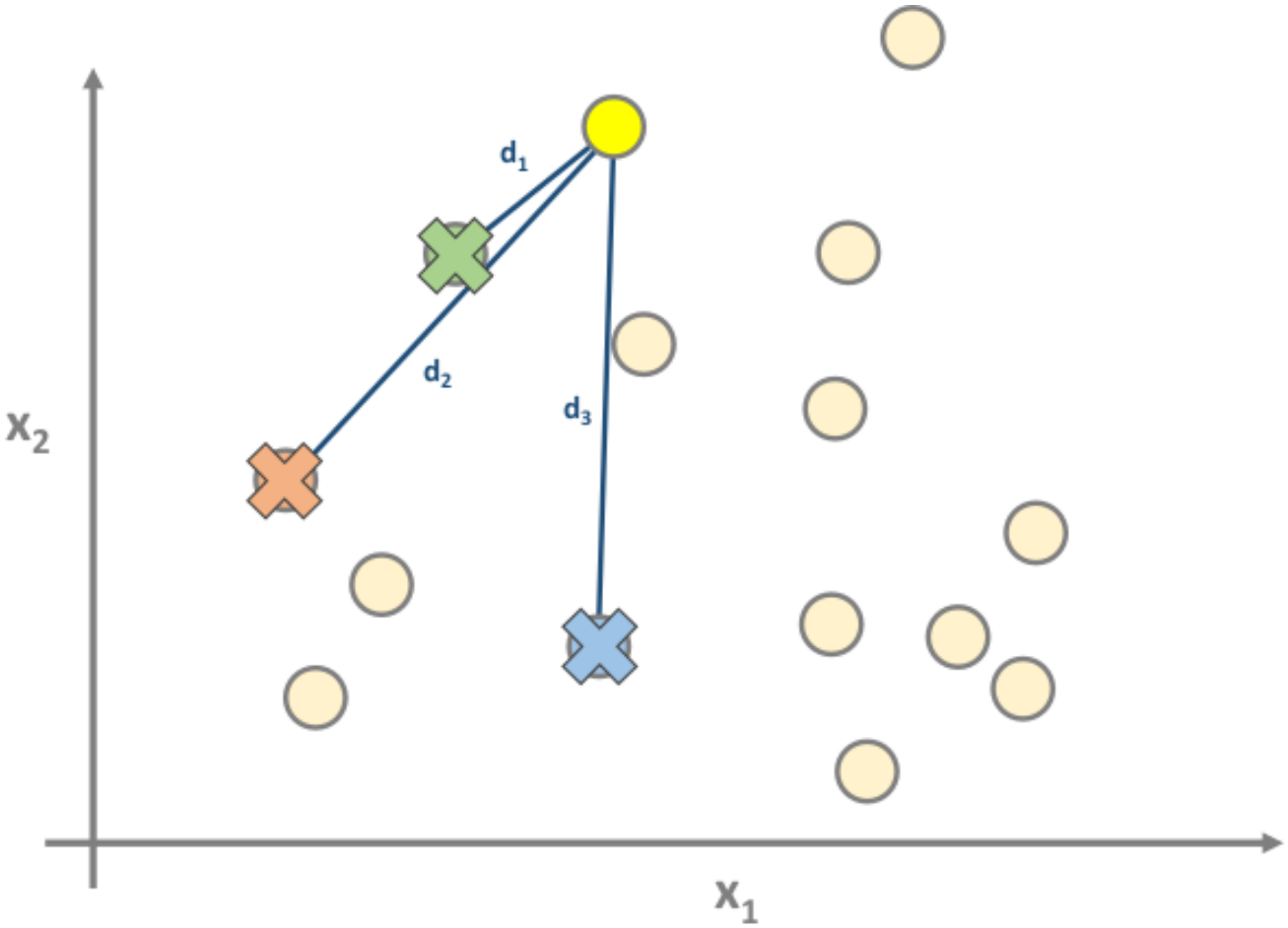
Example



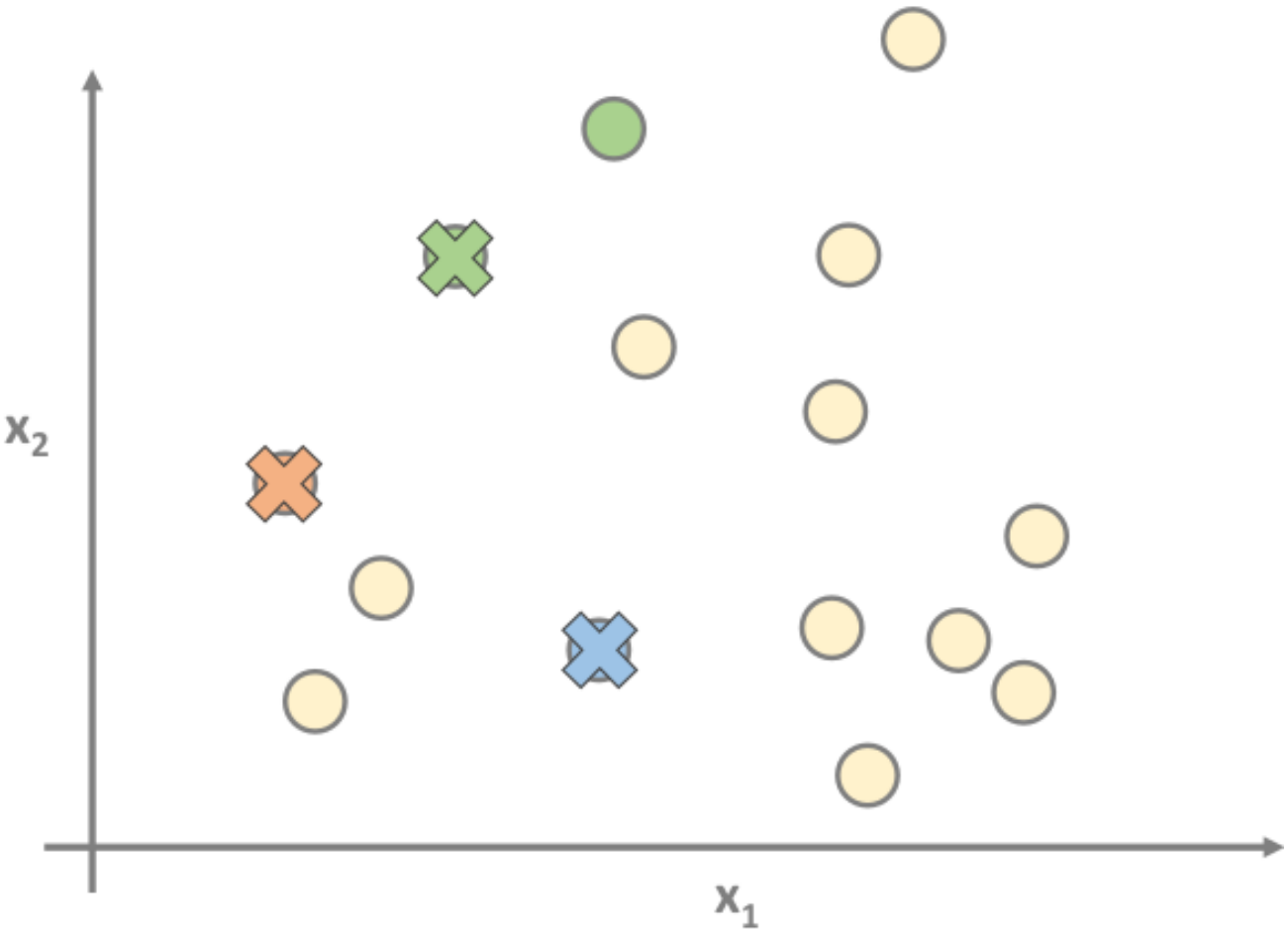
Example



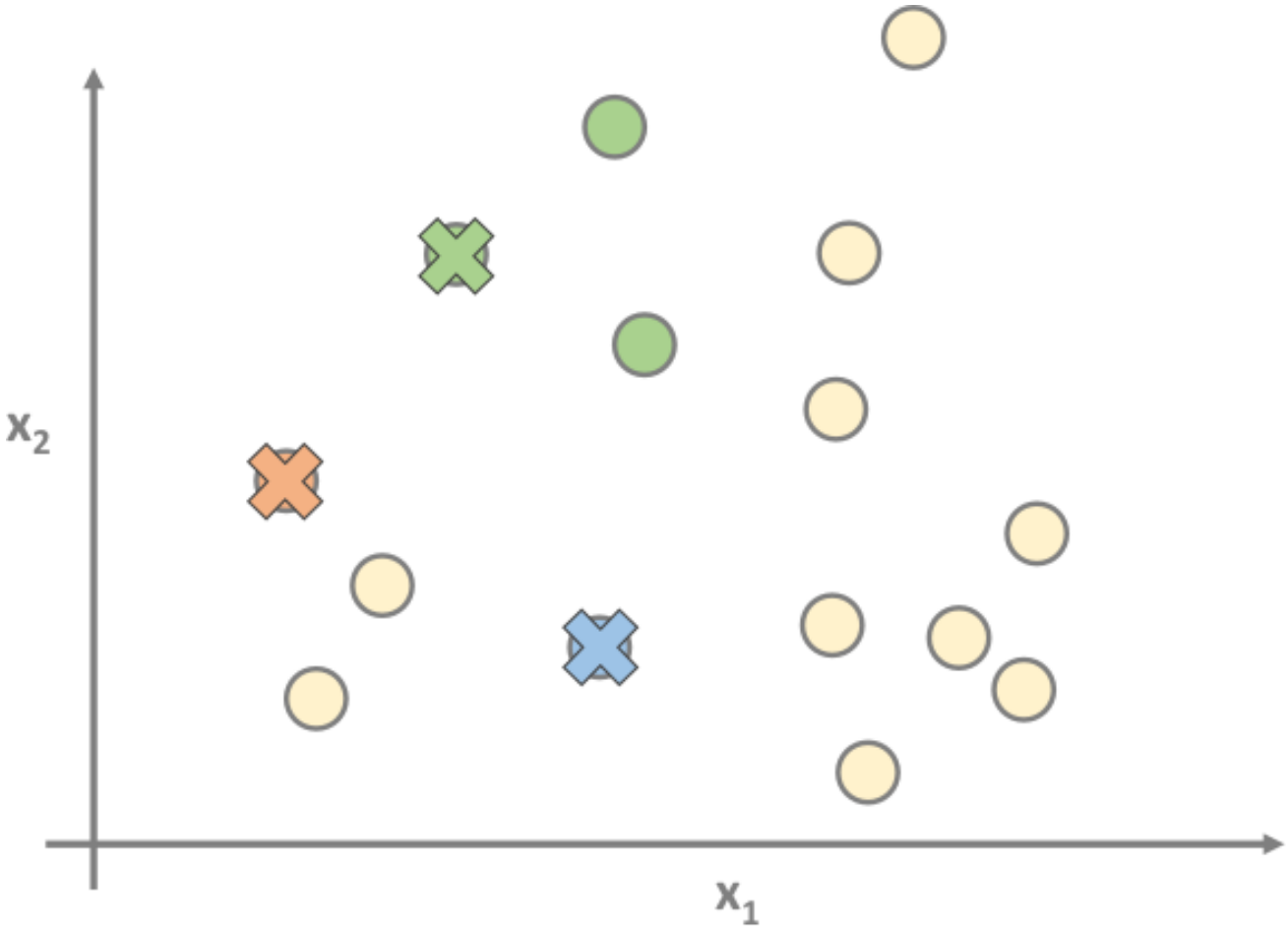
Example



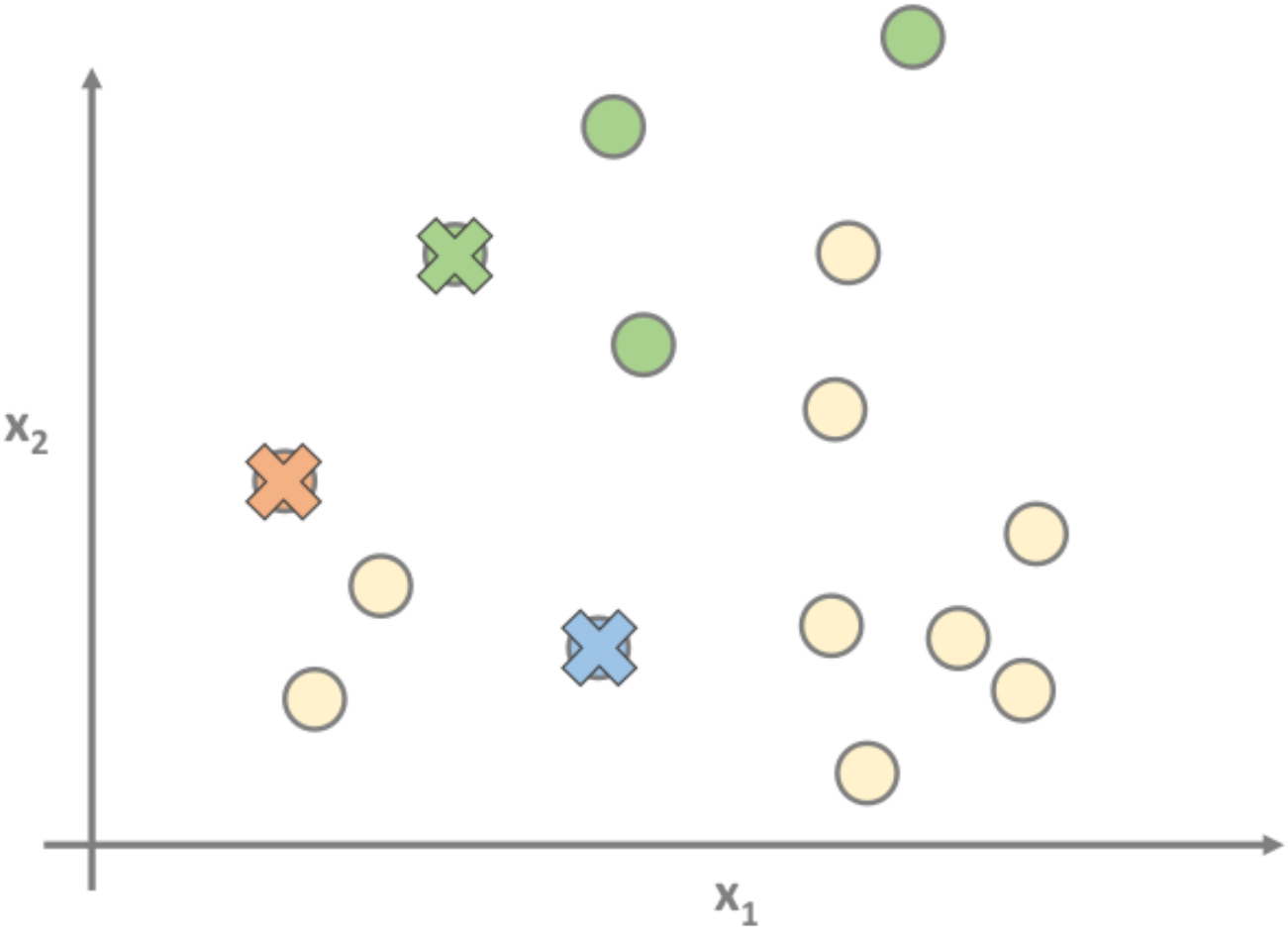
Example



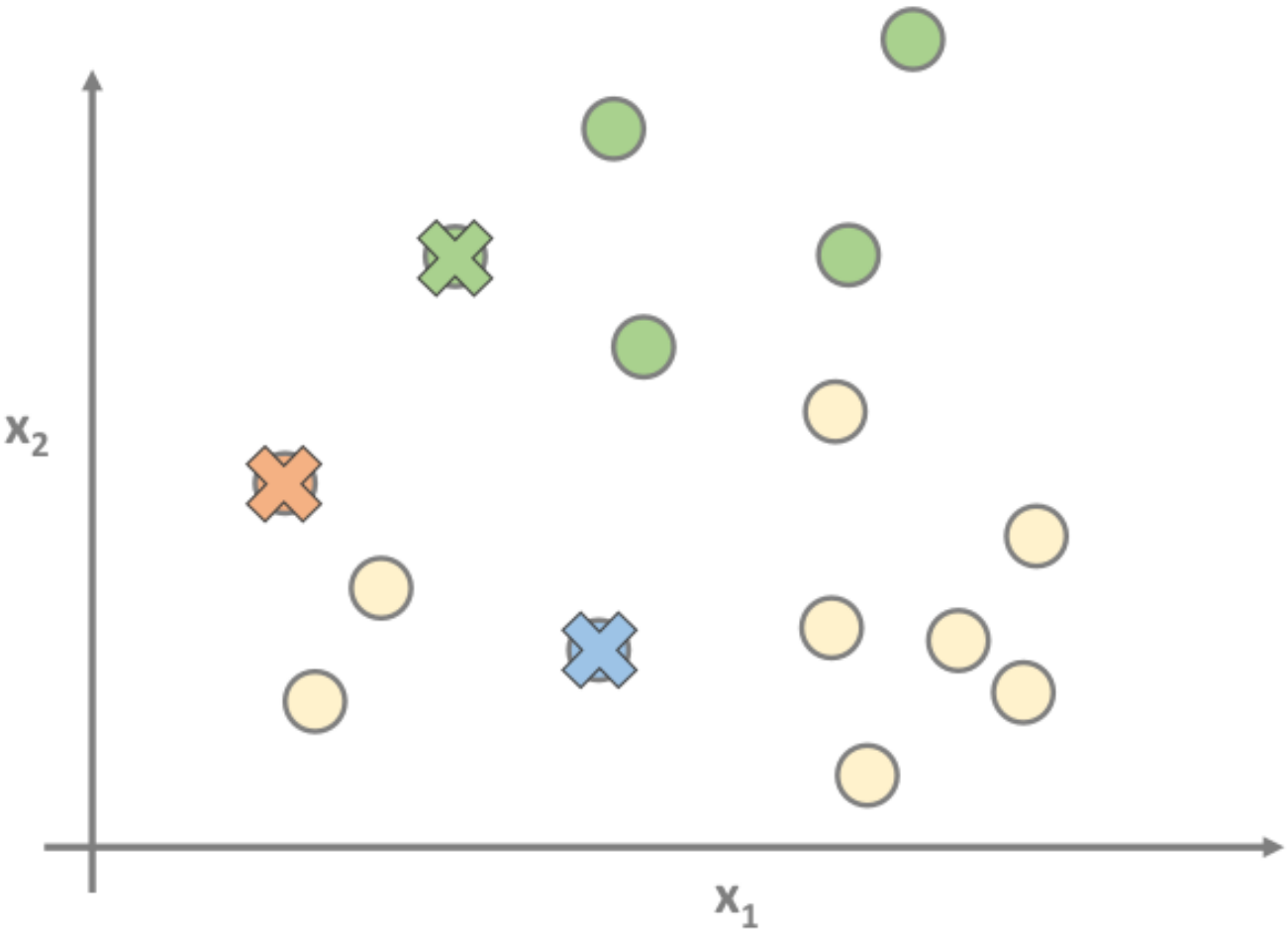
Example



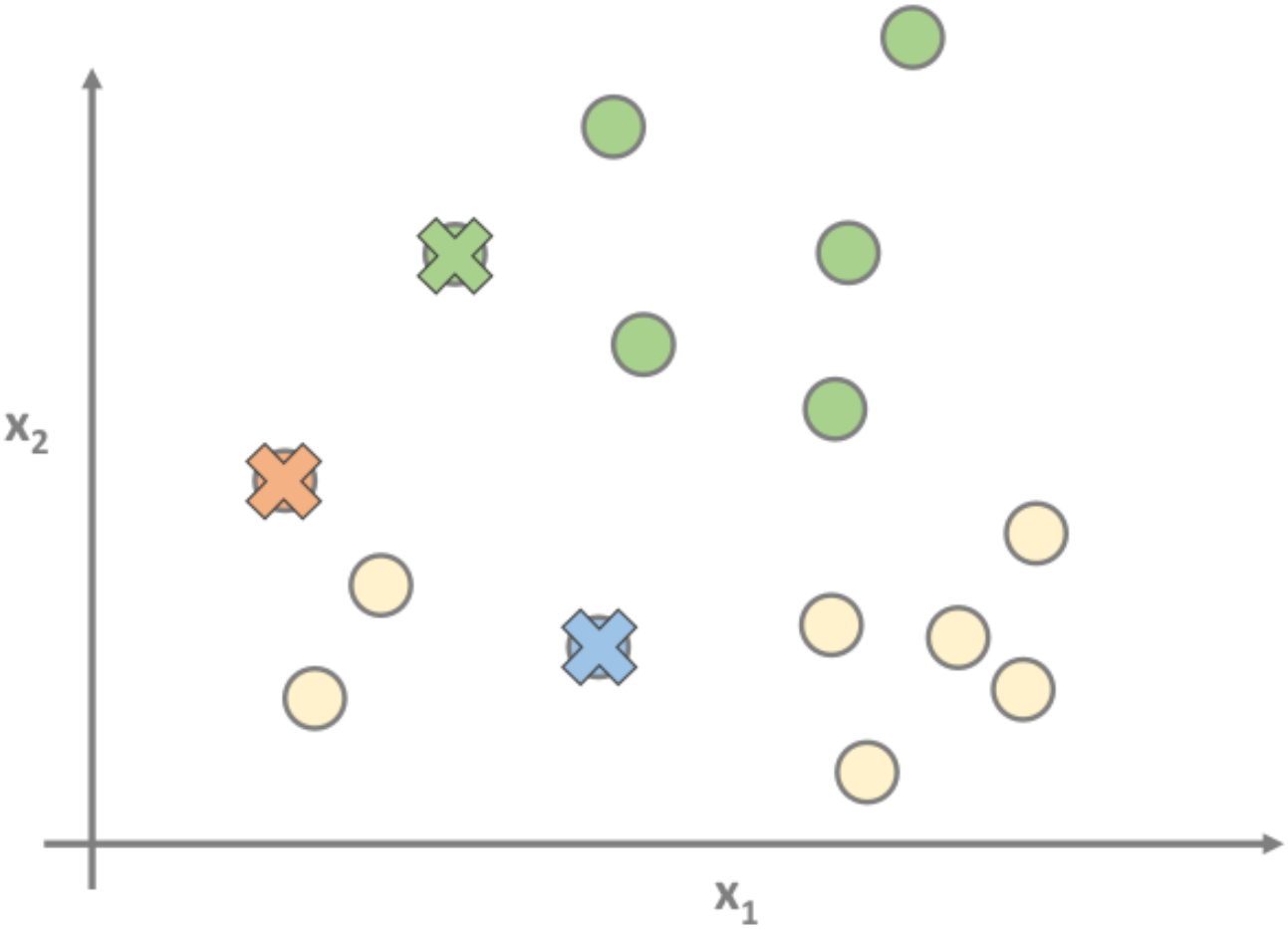
Example



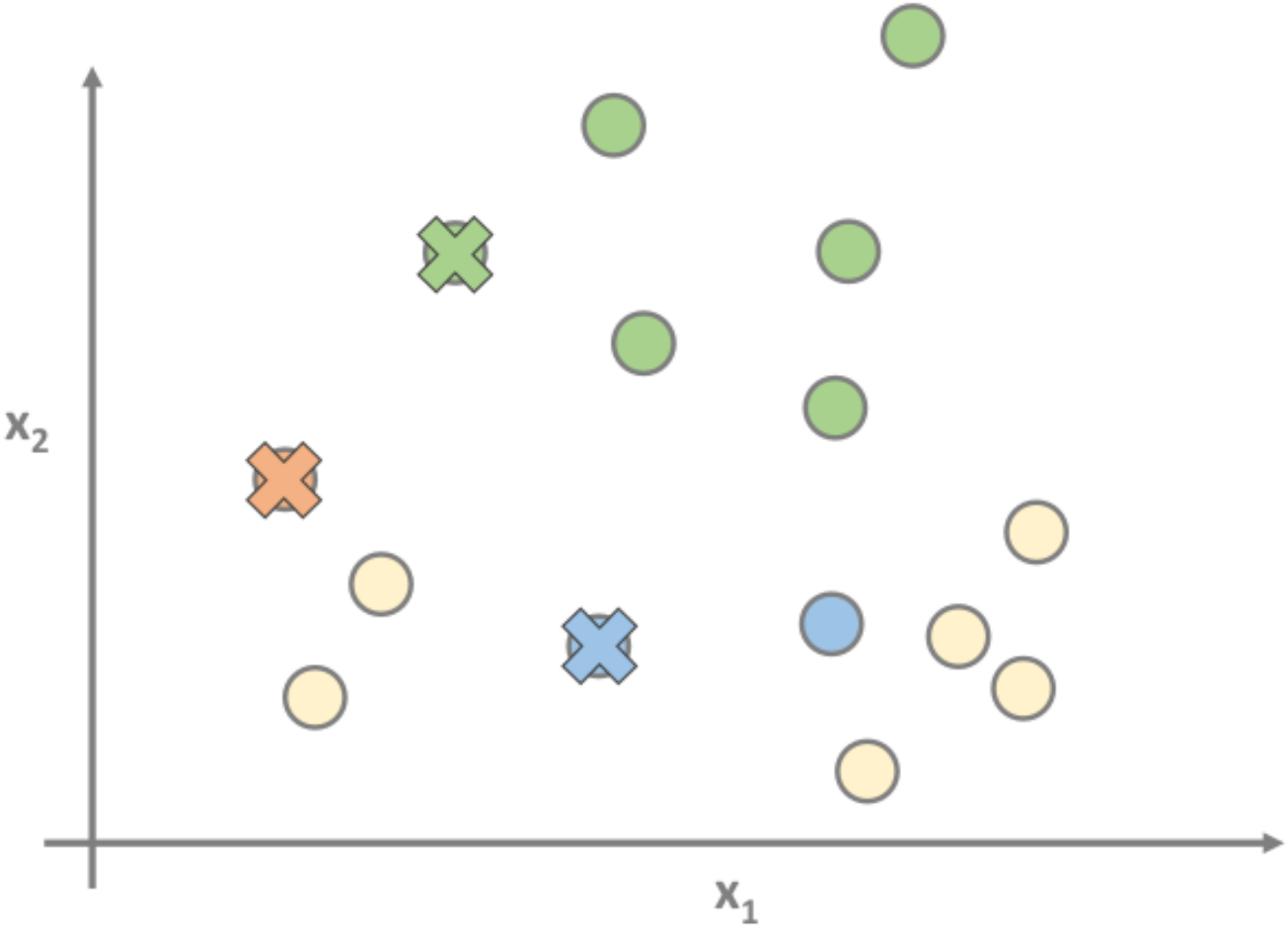
Example



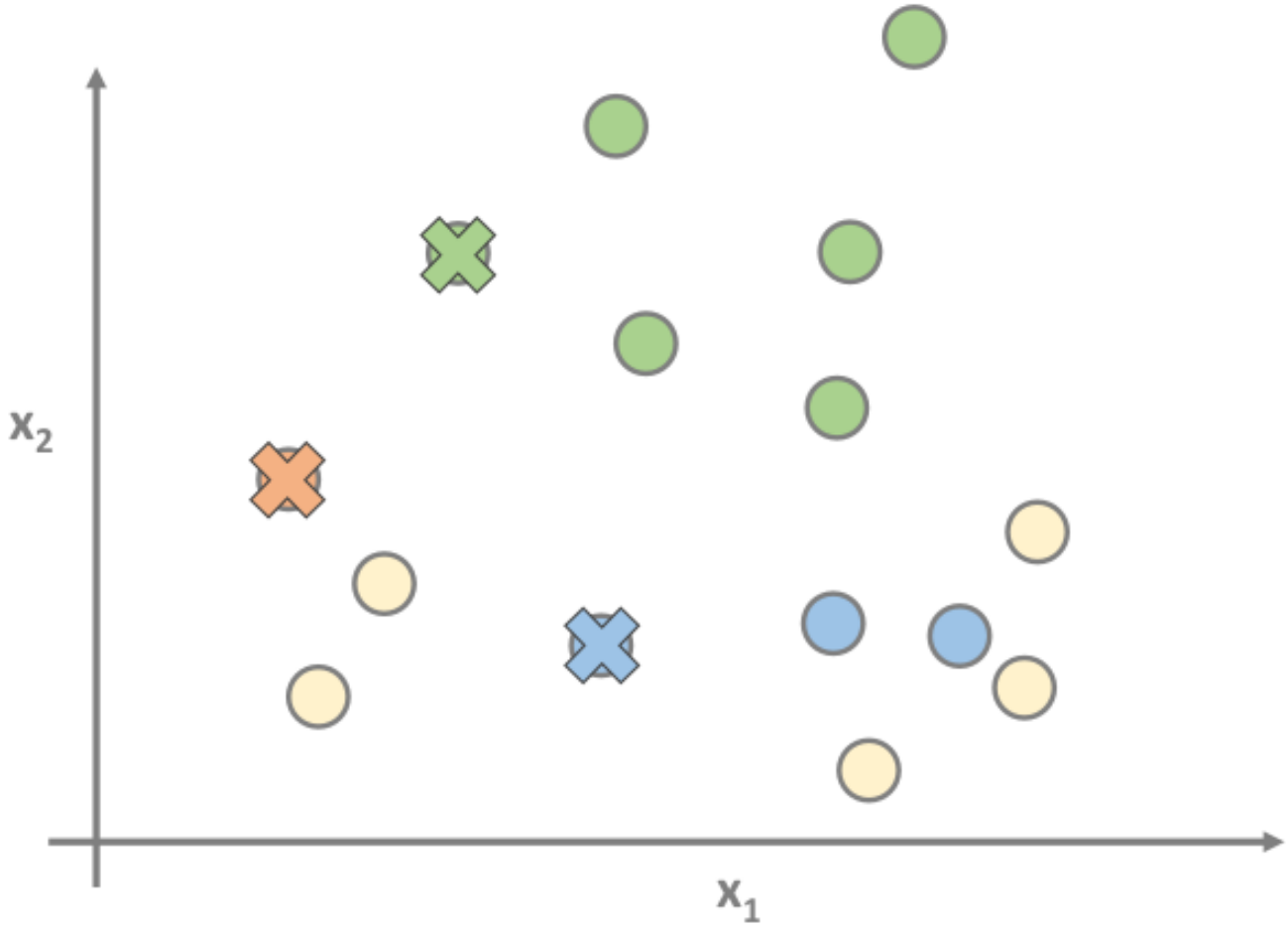
Example



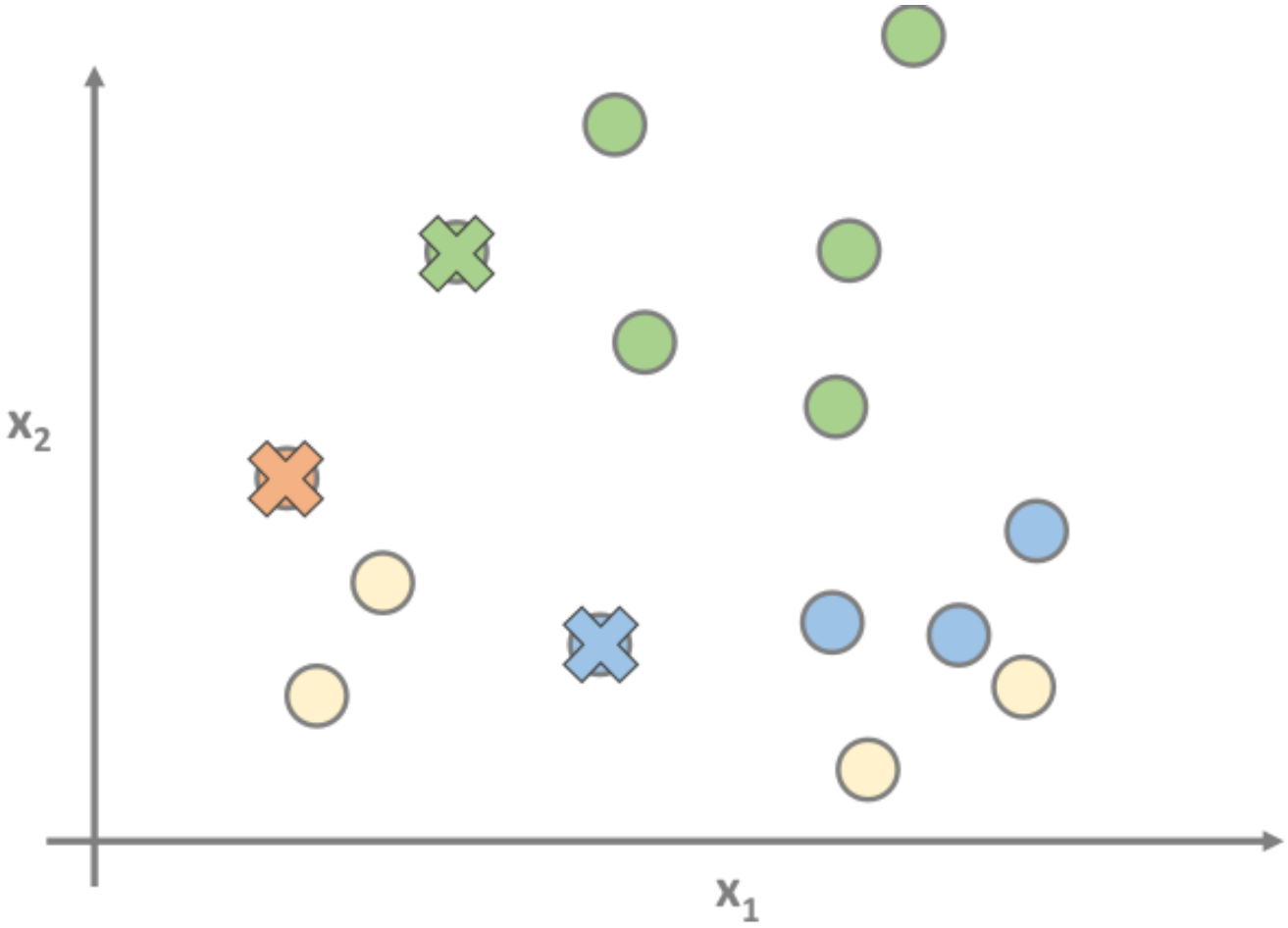
Example



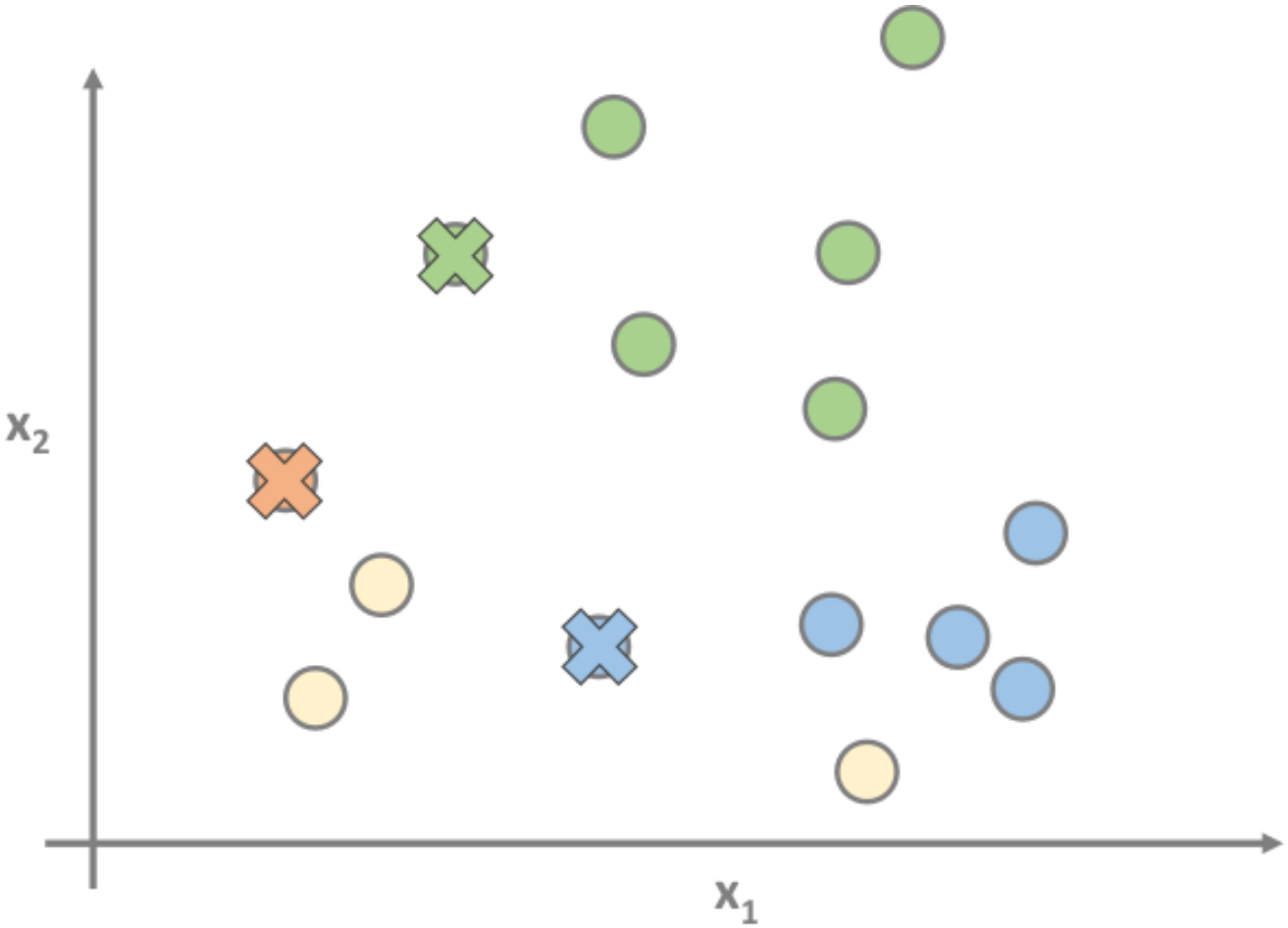
Example



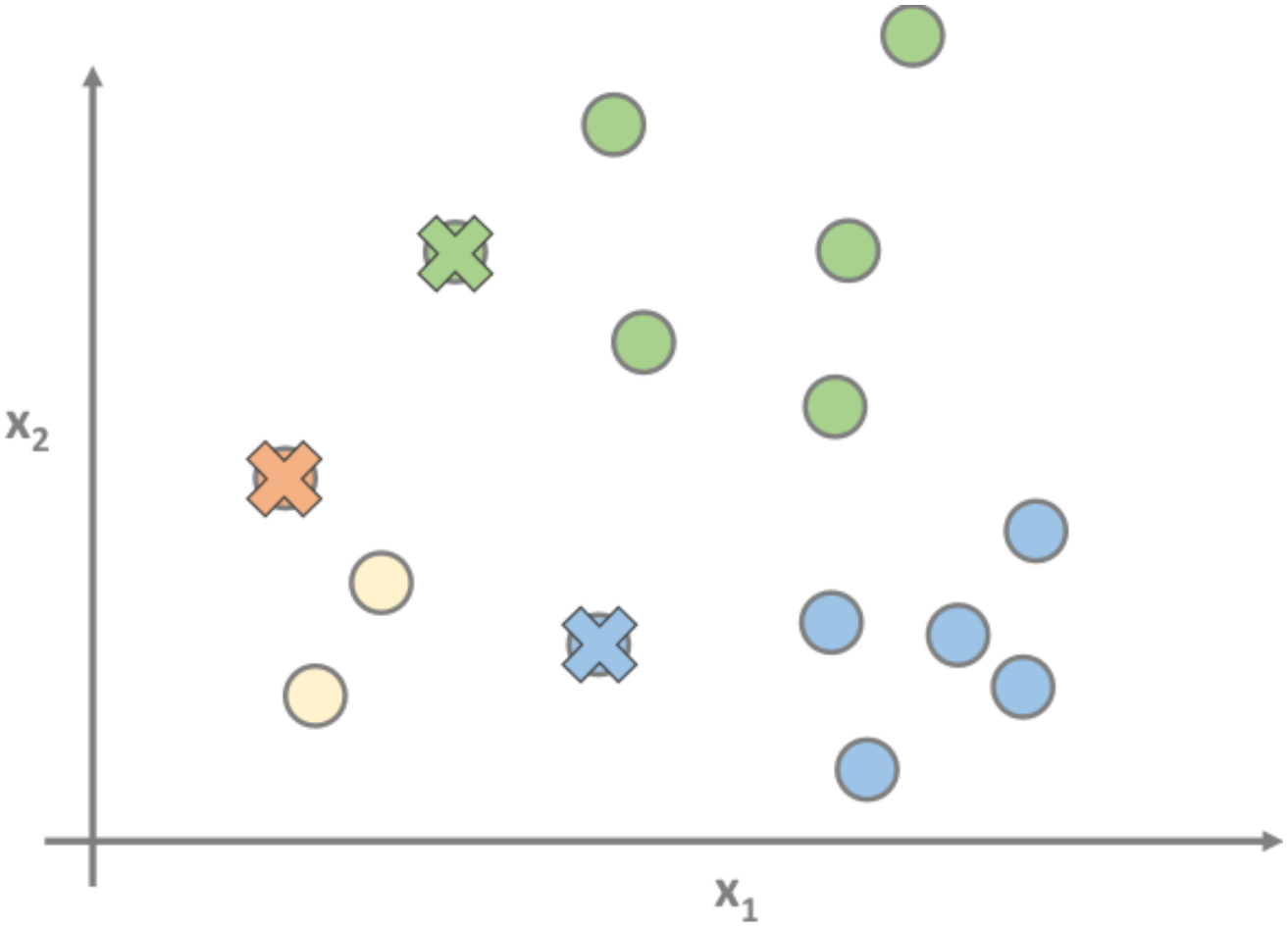
Example



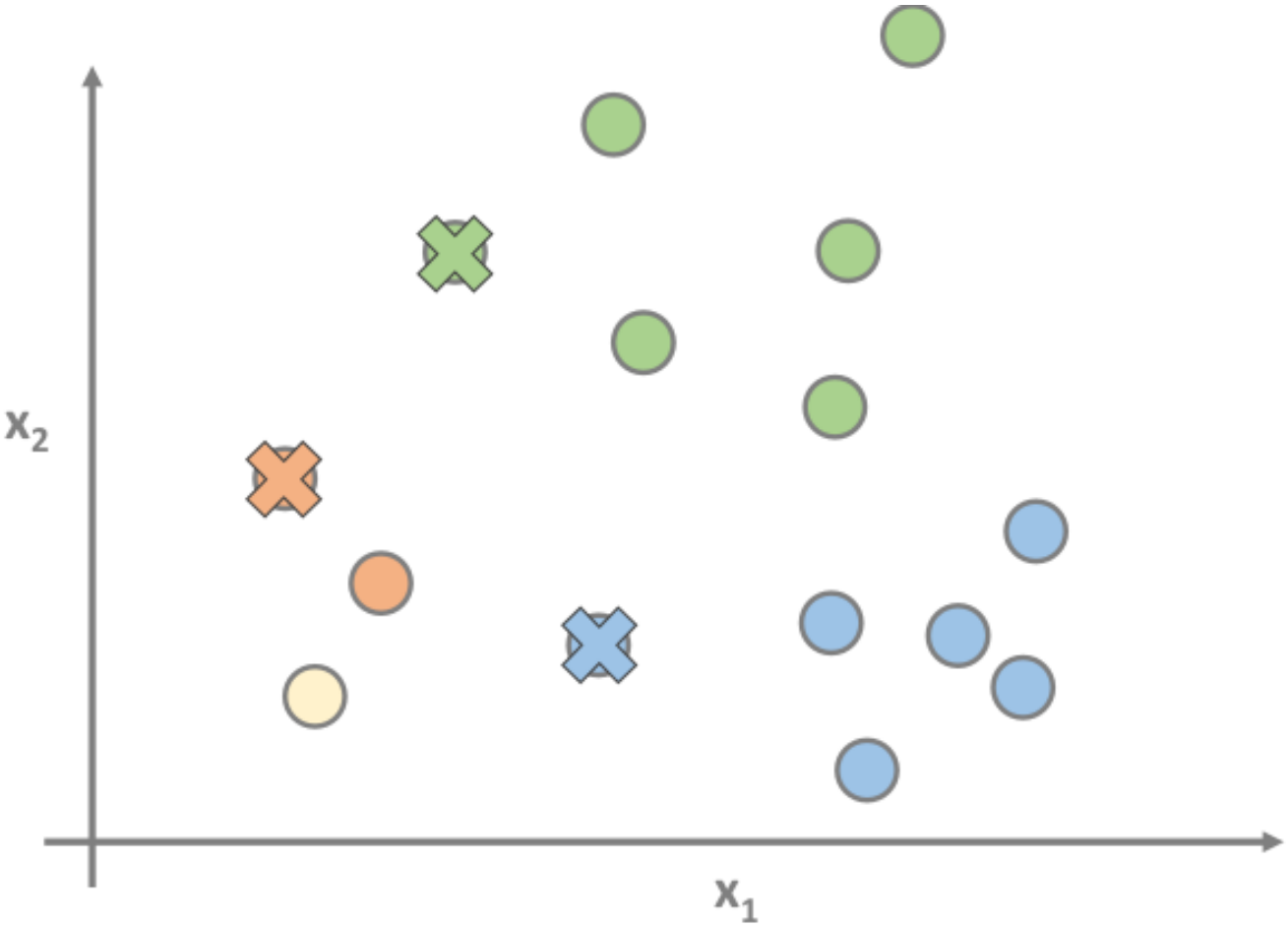
Example



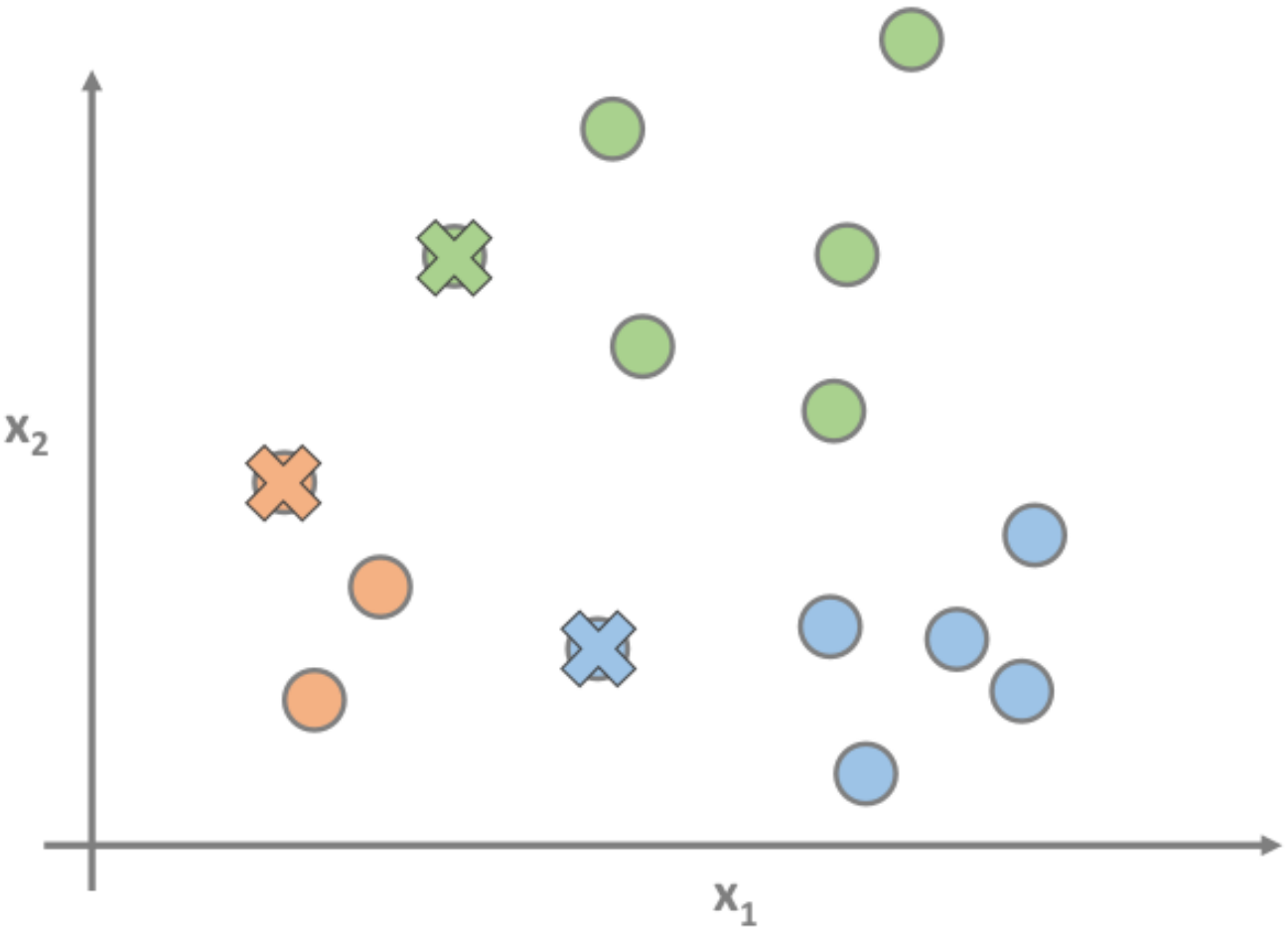
Example



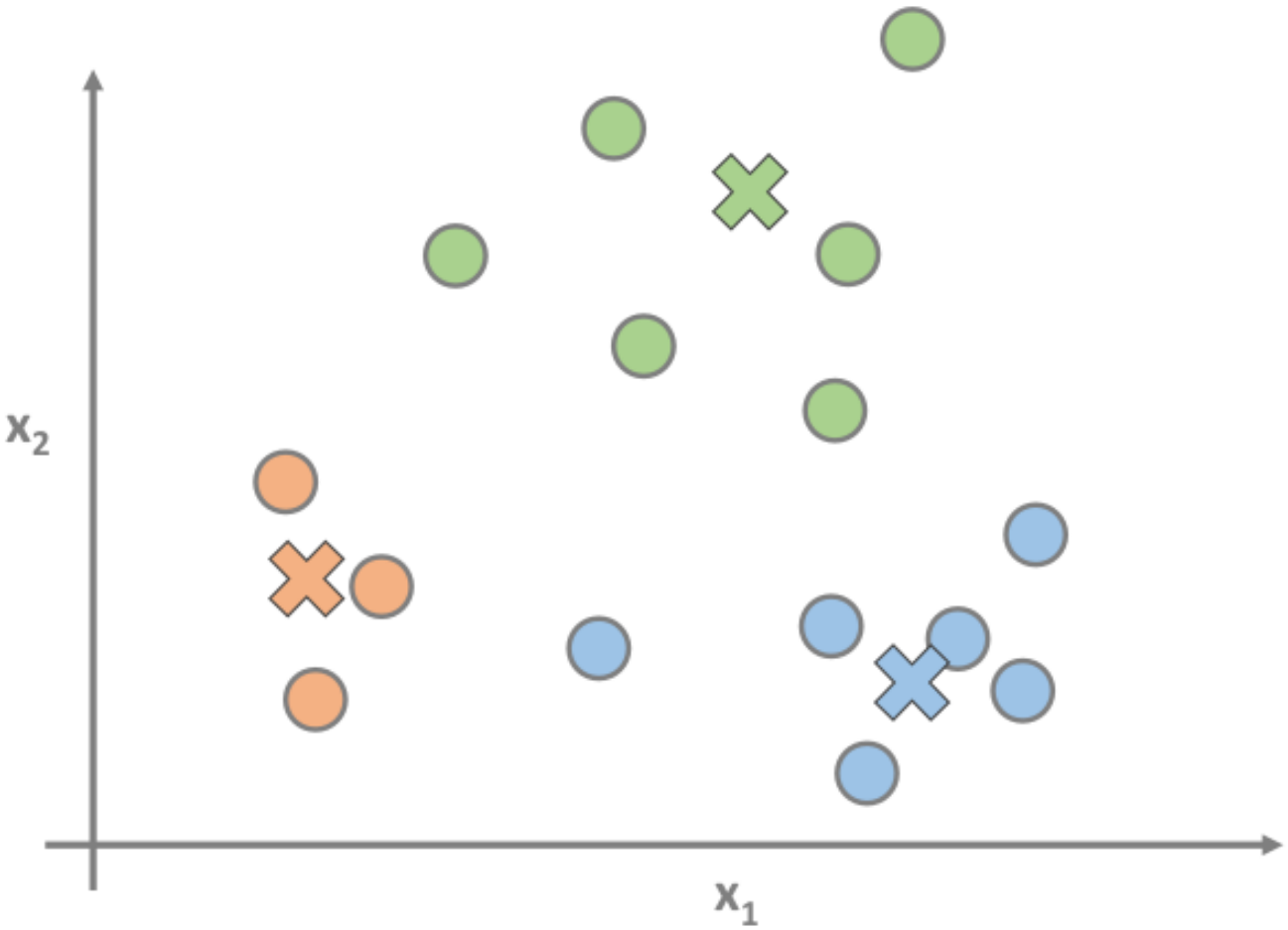
Example



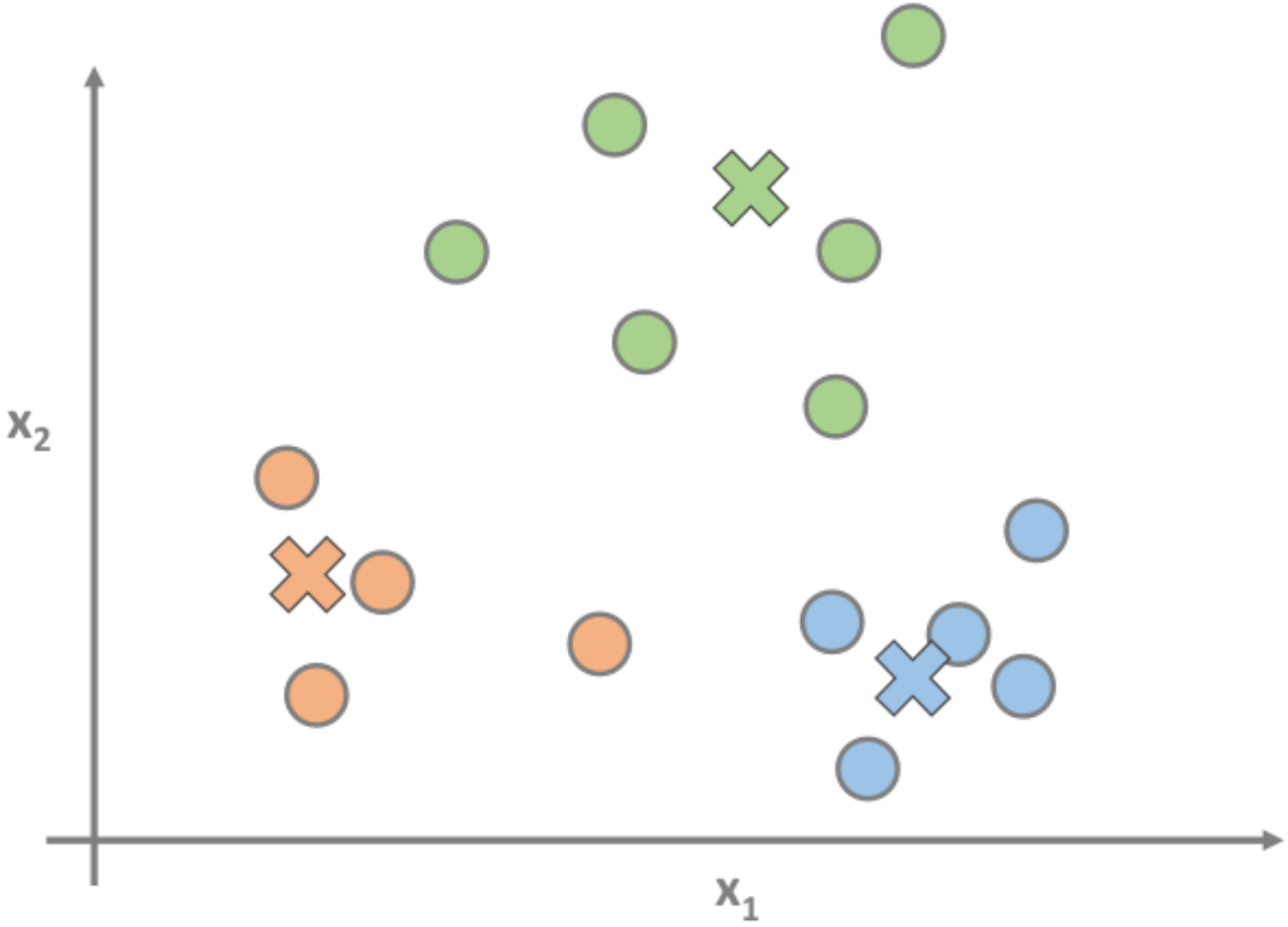
Example



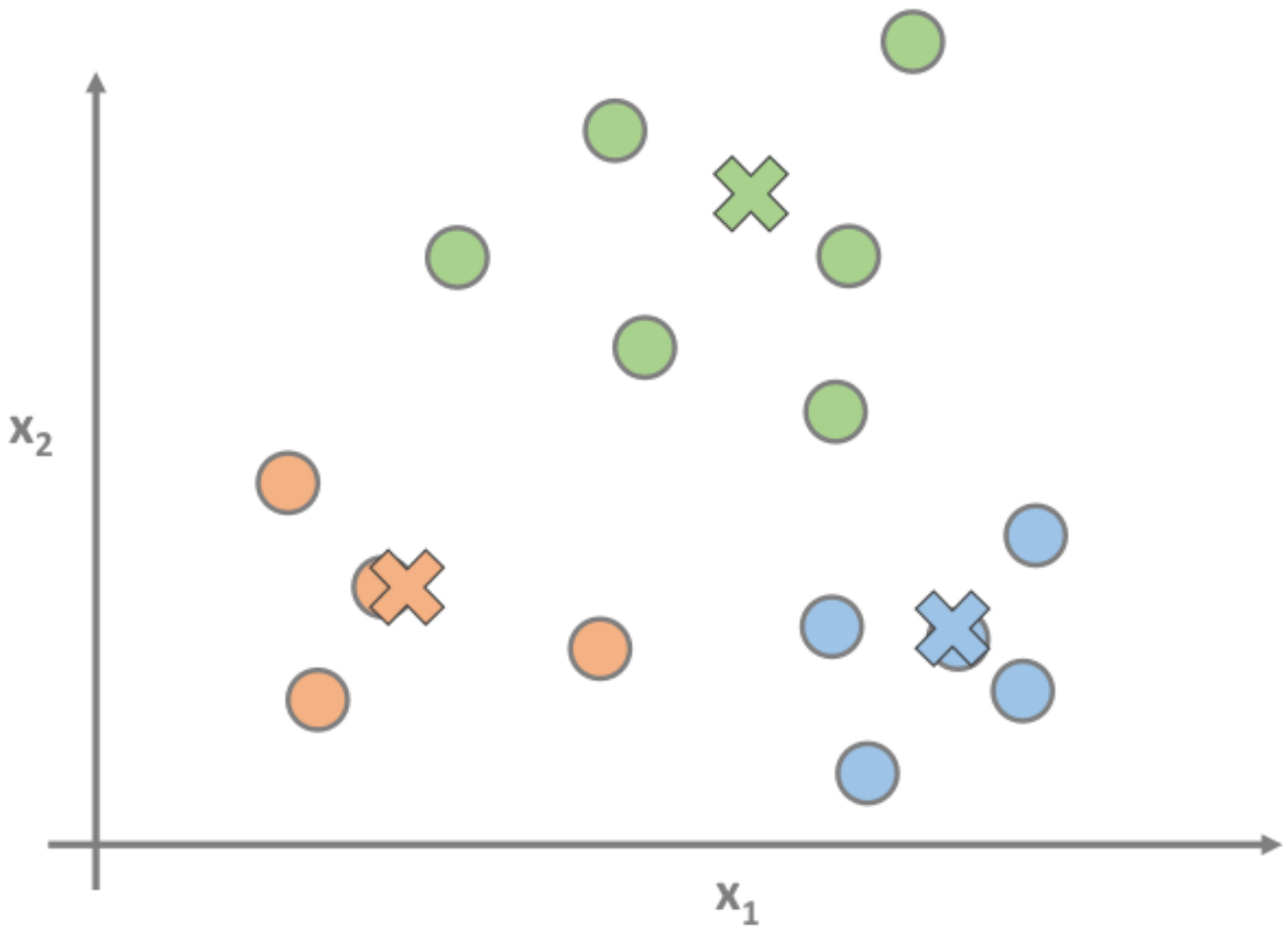
Example



Example



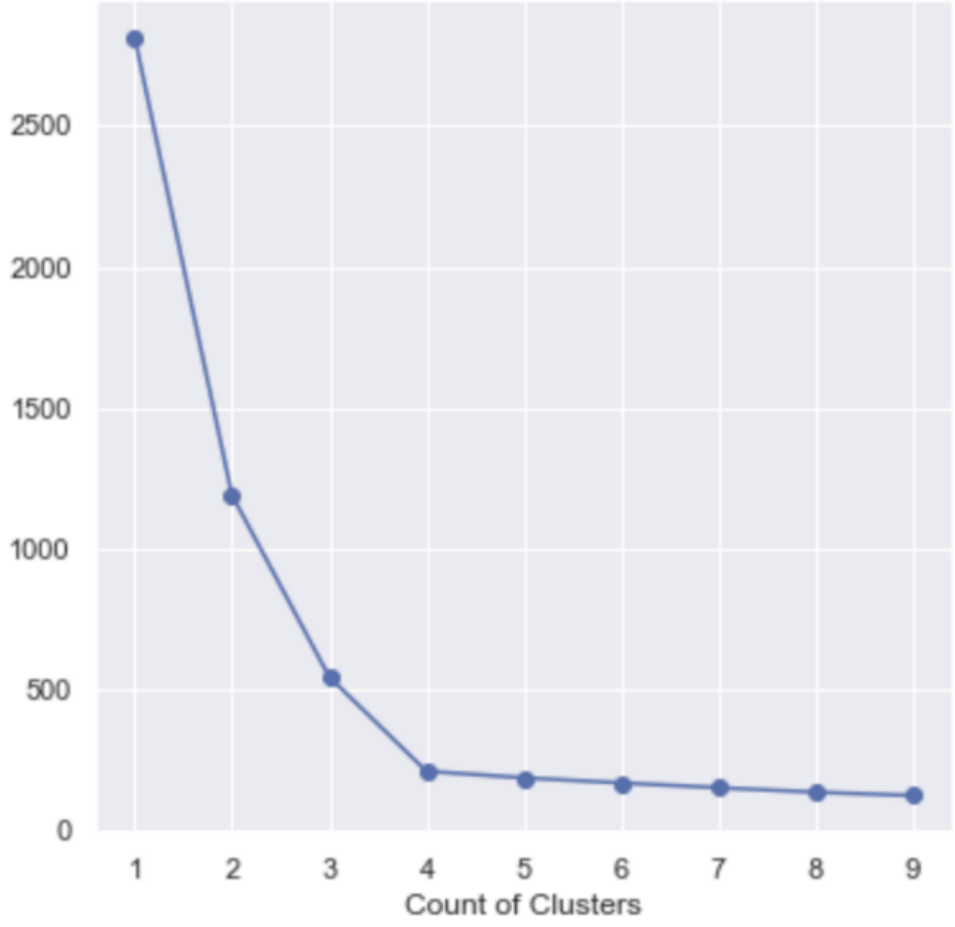
Example



How to find the best value of k?

- sometimes we have some a priori knowledge that we know how many clusters there are
- without any a priori knowledge: k is approximately equal to the square root of $N/2$
- **Elbow method**: we monitor the **within-cluster sum of squares (WCSS)** for different k values
  $WCSS_k = \sum_{i=1}^{k} \sum_{x \in C_i} \|x - \mu_i\|^2$
- we choose that k where the elbow is, or mathematically one should choose a number of clusters so that adding another cluster doesn't give much better modeling of the data or
- the WCSS won't decrease significantly any more.

Example



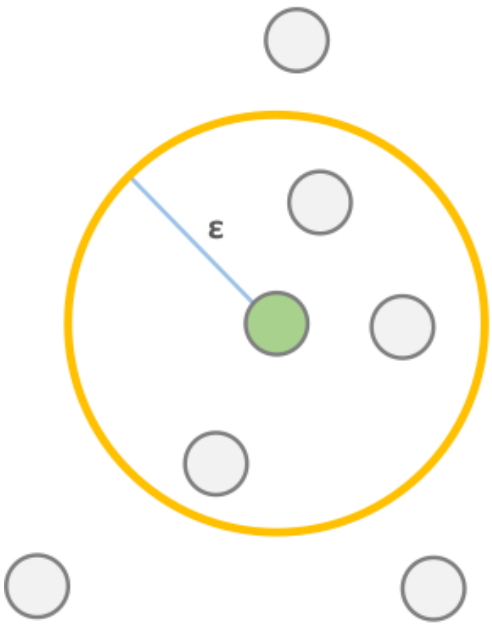
Pros and contras

Advantages	**Disadvantages**
- very simple model	- it is not guaranteed to find the optimal set of clusters
- we have to know somehow the k parameter	- guranteed convergence

DBSCAN

- it is a **clustering algorithm** from 1996
- Density-Based Spatial Clustering of Applications with Noise
- there are two parameters (ε and K)
- very good for searching outliers and noise

DBSCAN



- the **neighbors** of a point are that points which are closer to it than ε
- a point is called **core point** if it has at least K neighbors
- **boundary point**: not a core point, but it's a neigbor of a core point
- **noise**: neither a core point nor a boundary

DBSCAN

The algorithm

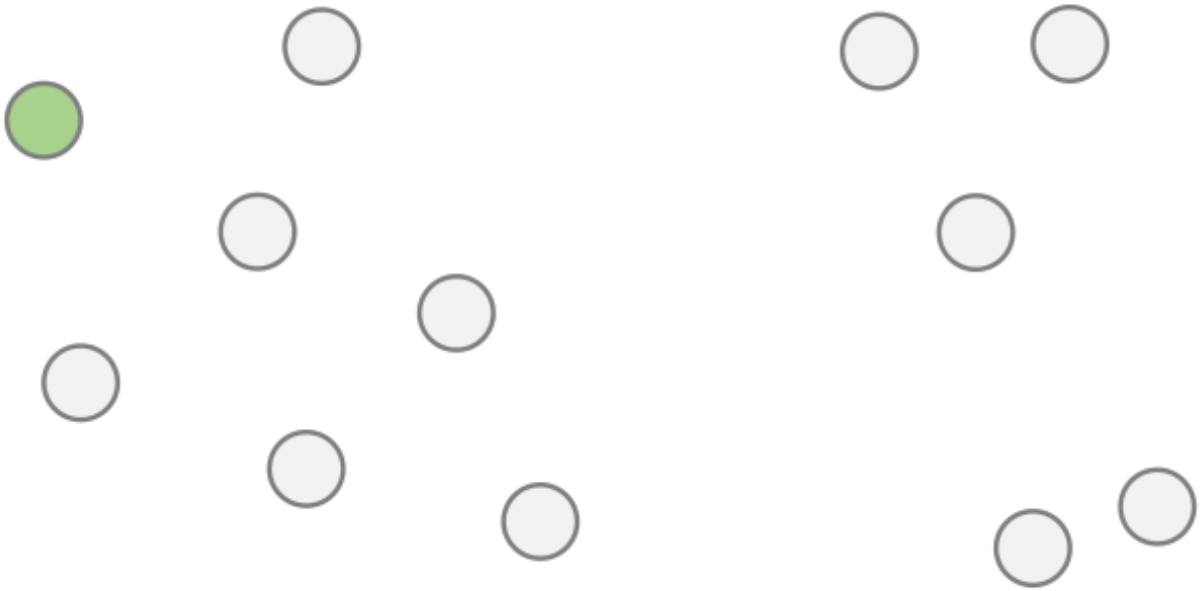
while there are any unassigned point

klaszterId=0

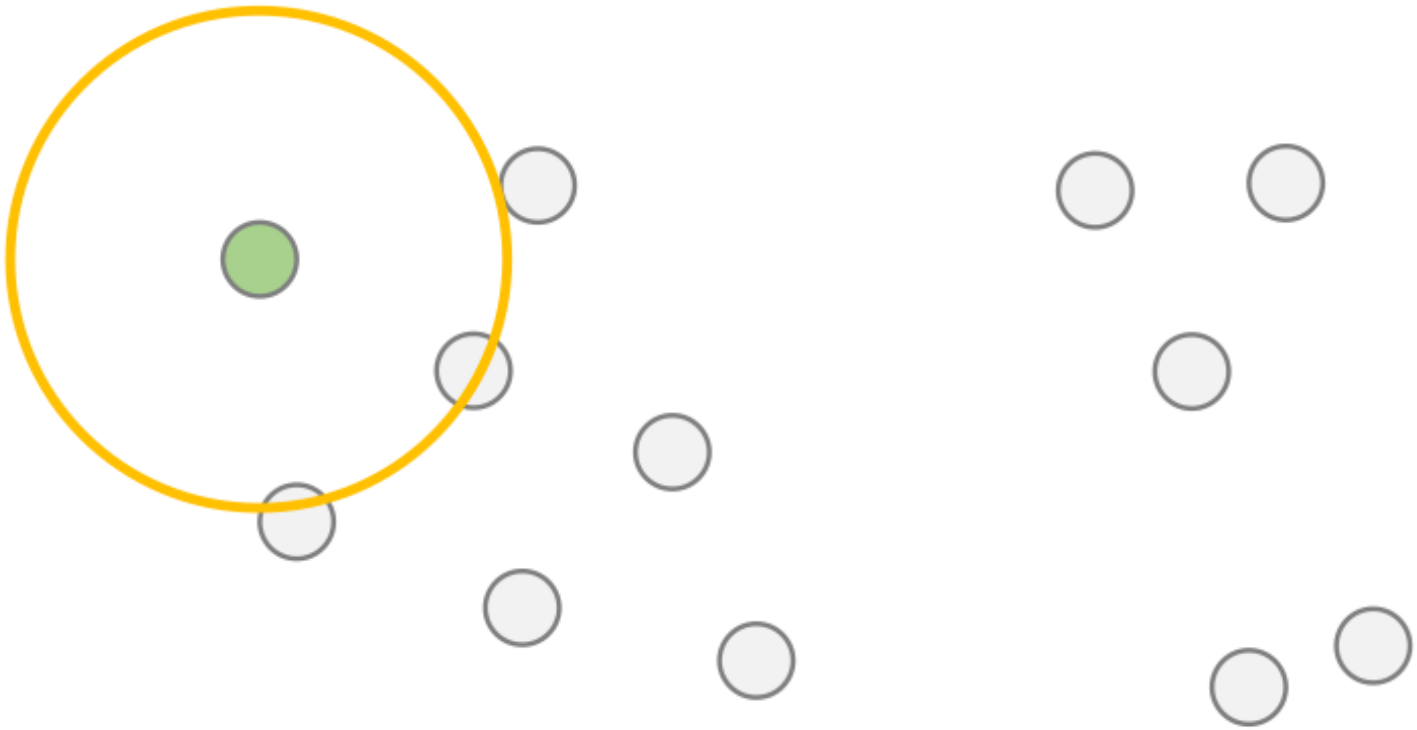
- take an unassigned point x
- if it is a core point then create a list: add x and all of its neighbors, and this is a **new cluster** - **klaszterId++**

  otherwise it is a **noise point**
- While the list is not empty:
  - take an element from the list, and if it is a core point then add all of its neighbors to the list, and remove the element from the list
  - we assign the removed element to the actual cluster (klaszterId)

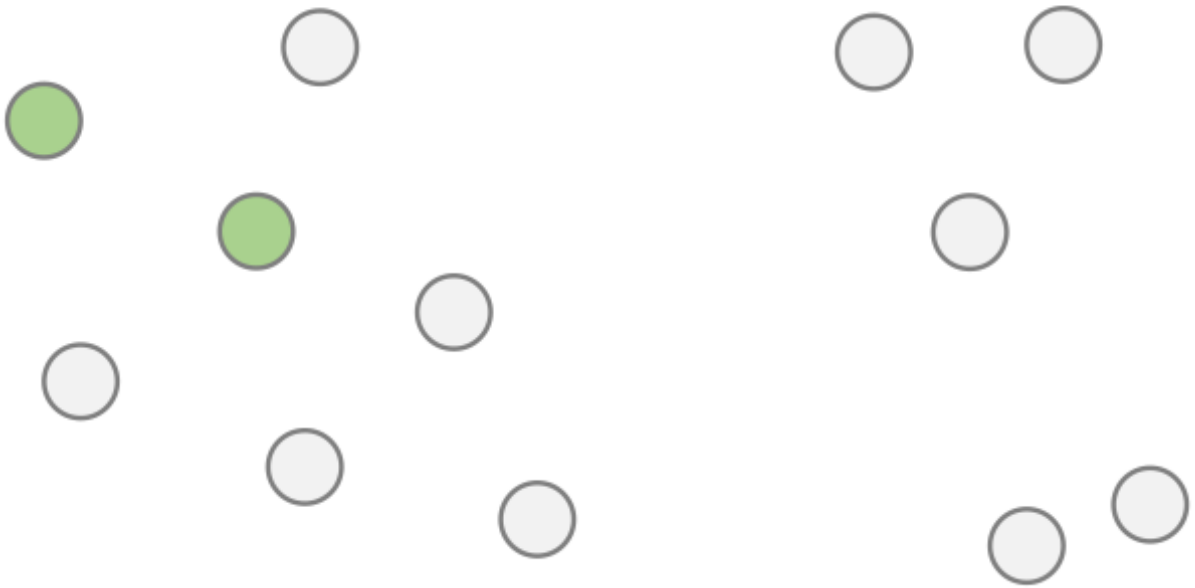
Example



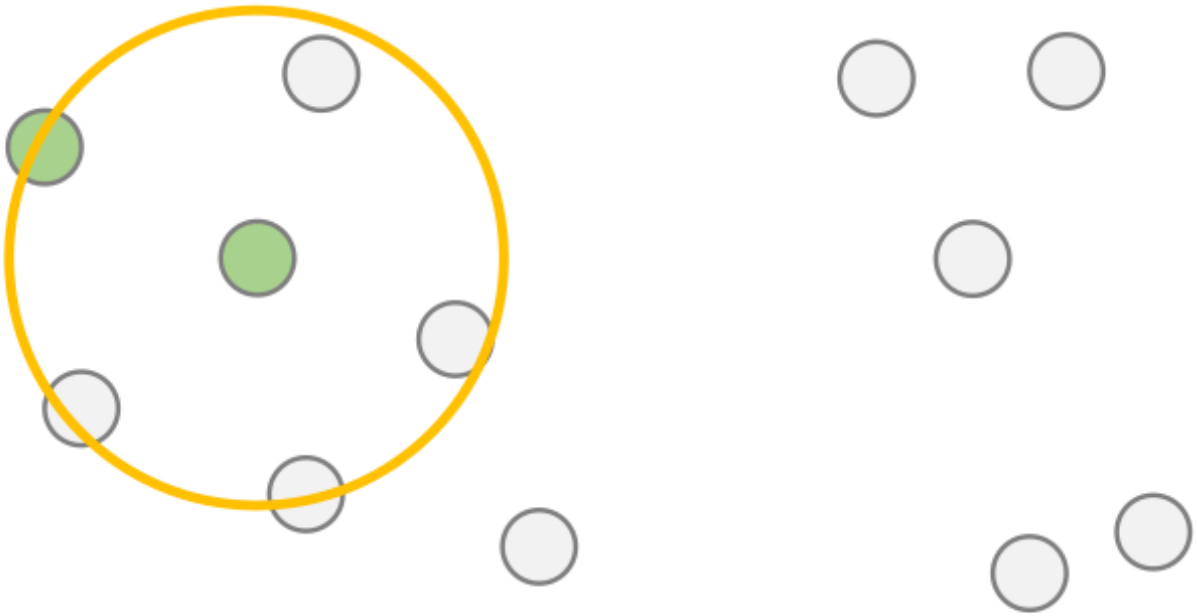
Example



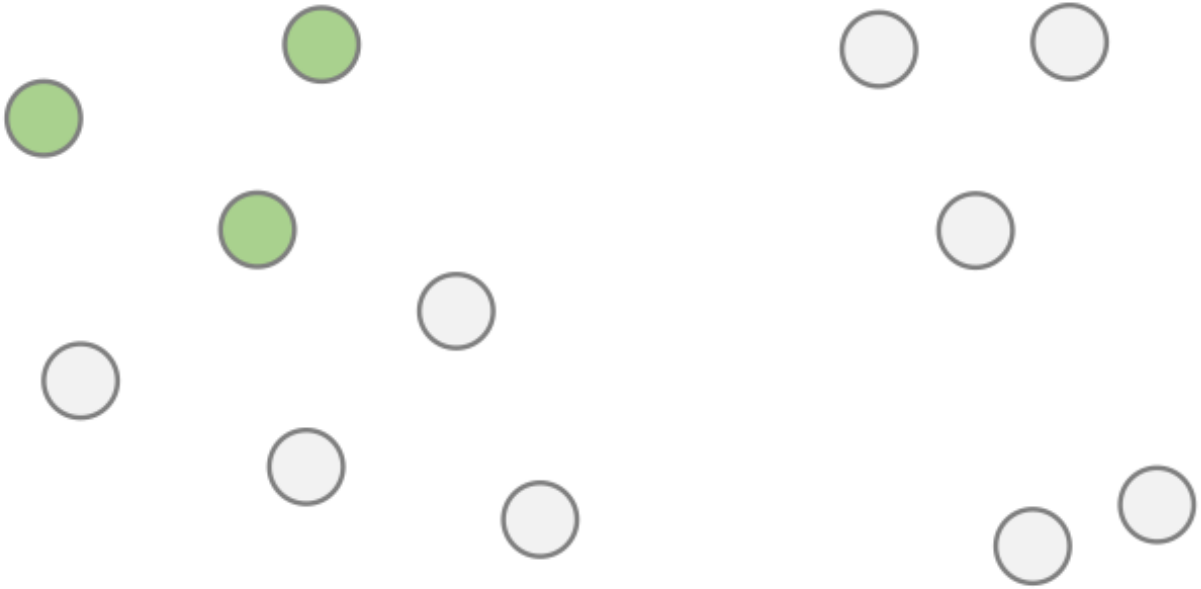
Example



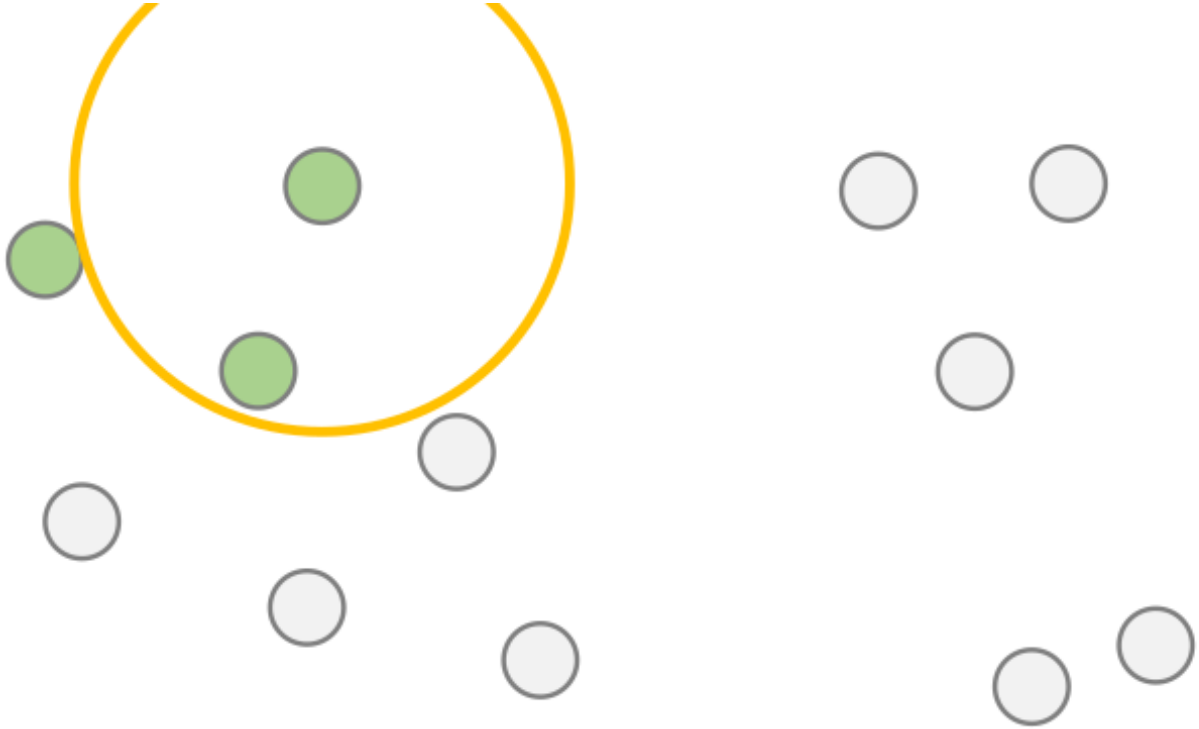
Example



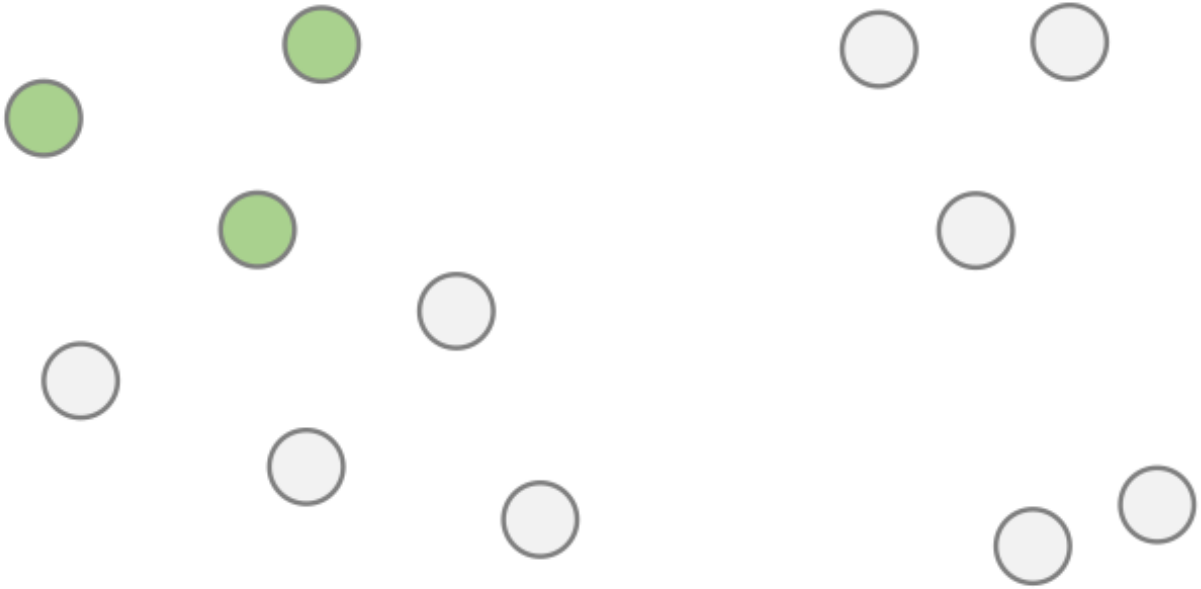
Example



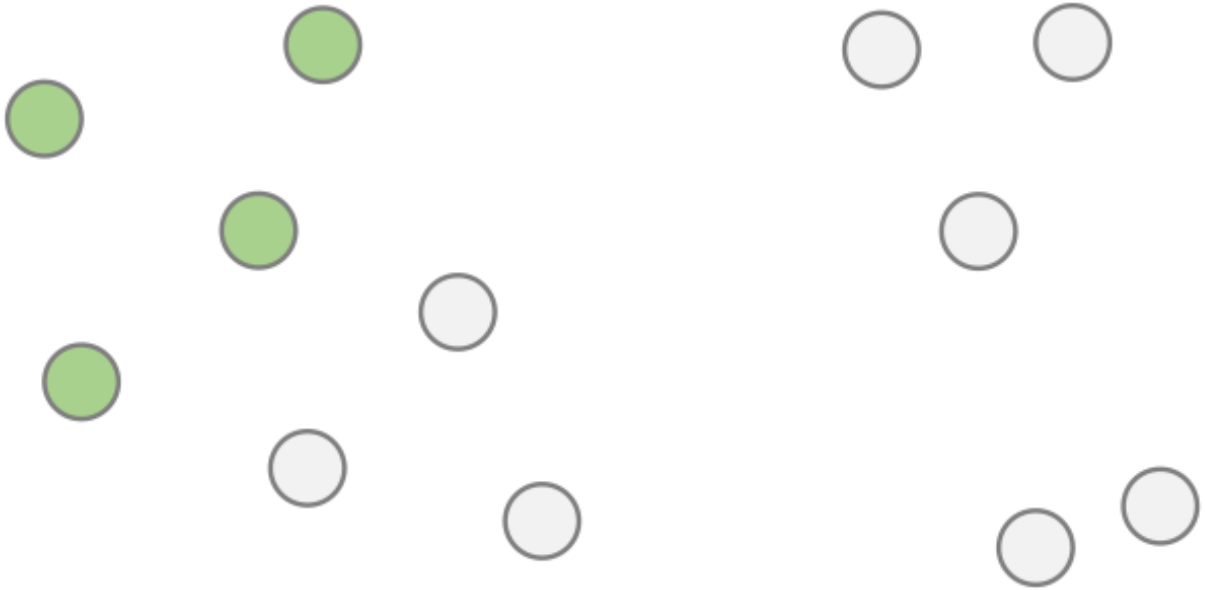
Example



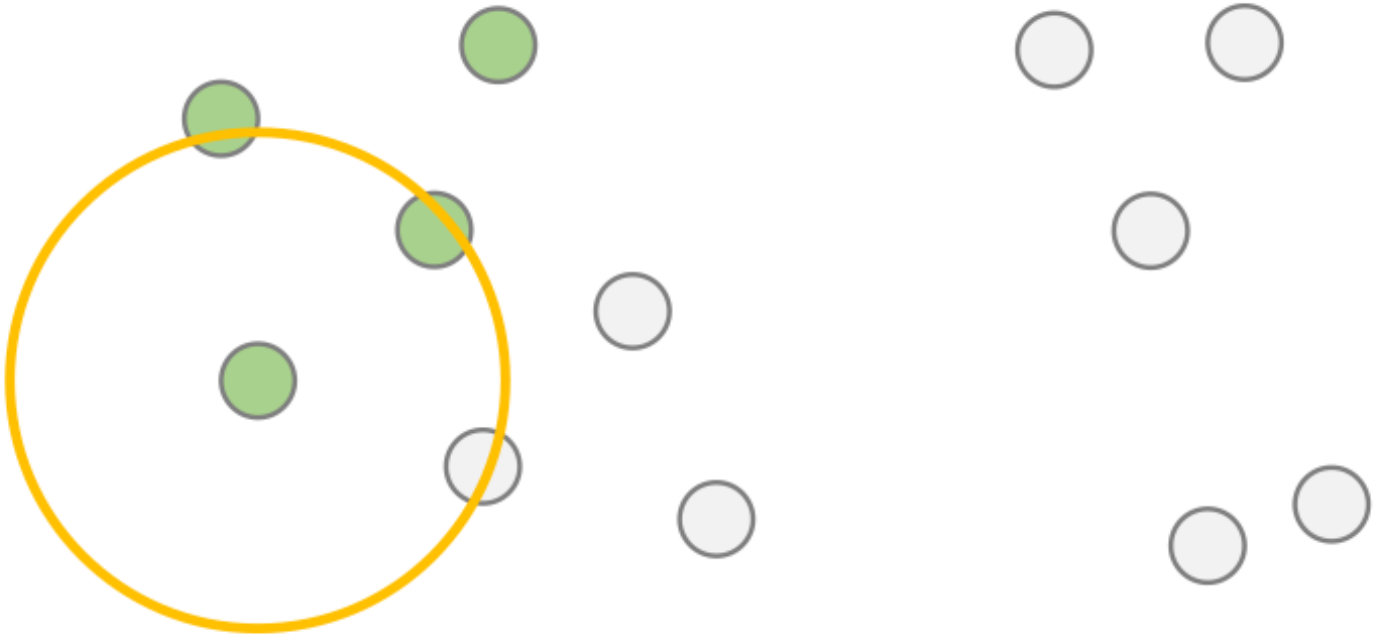
Example



Example

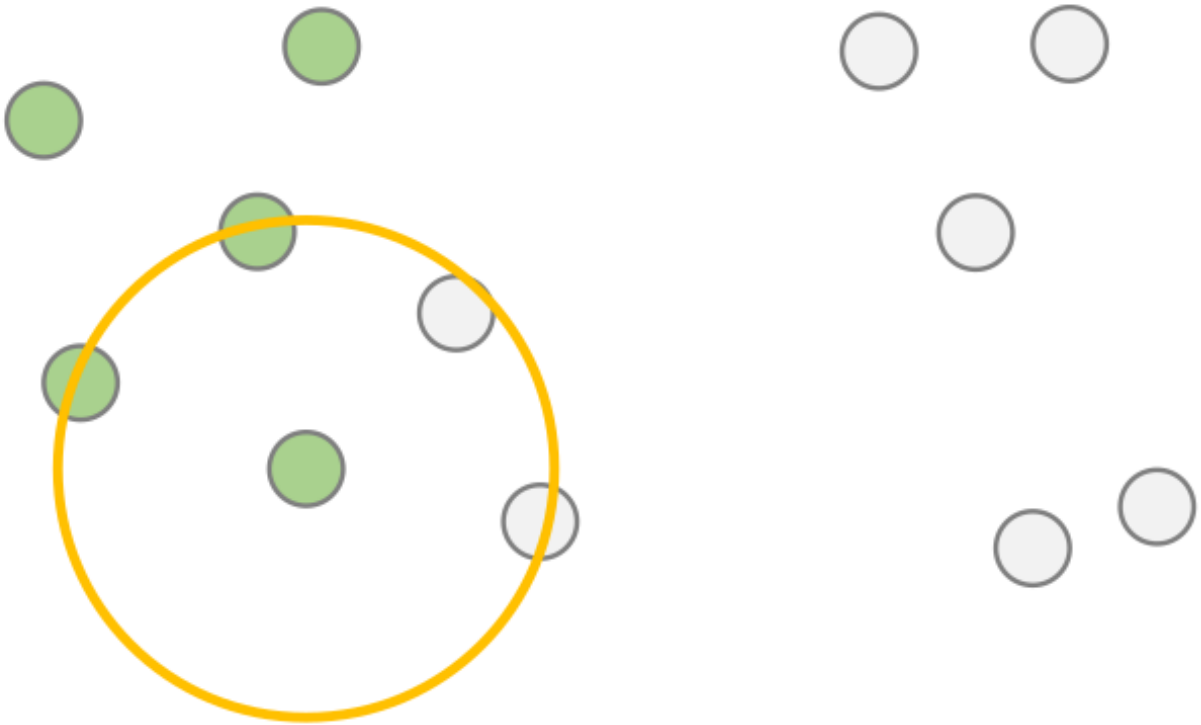


Example

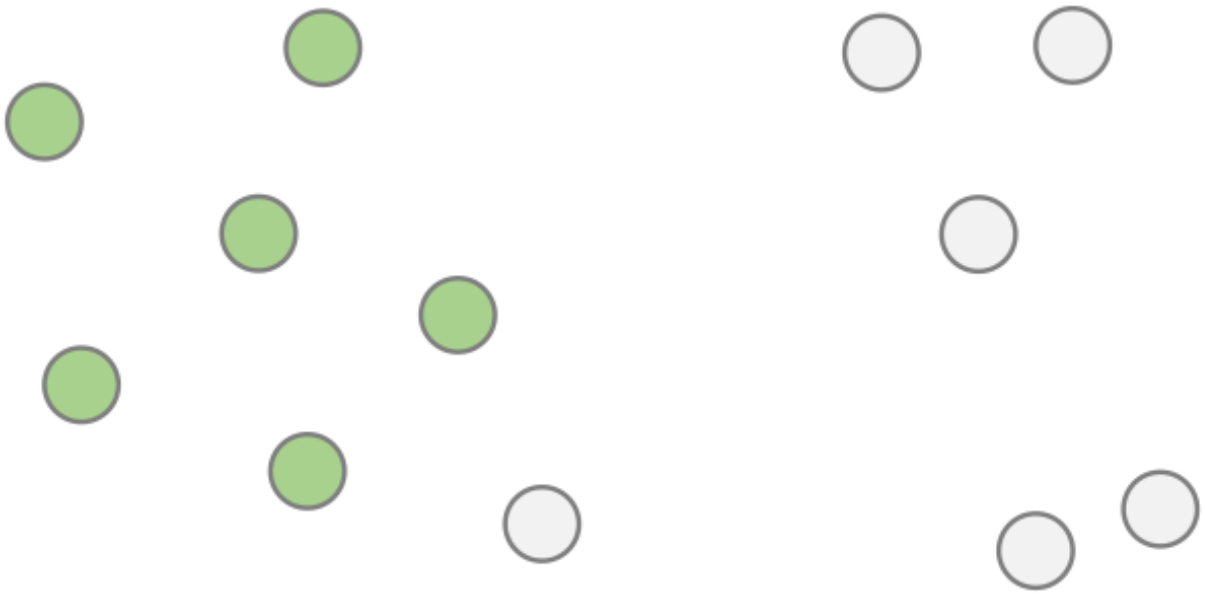


Example

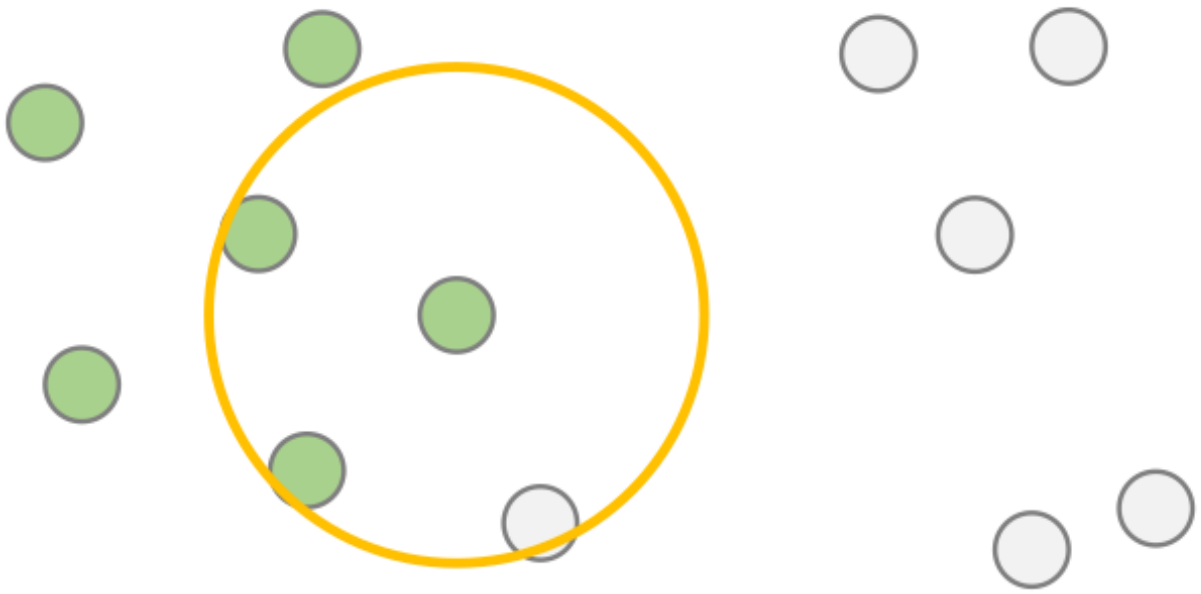
Example



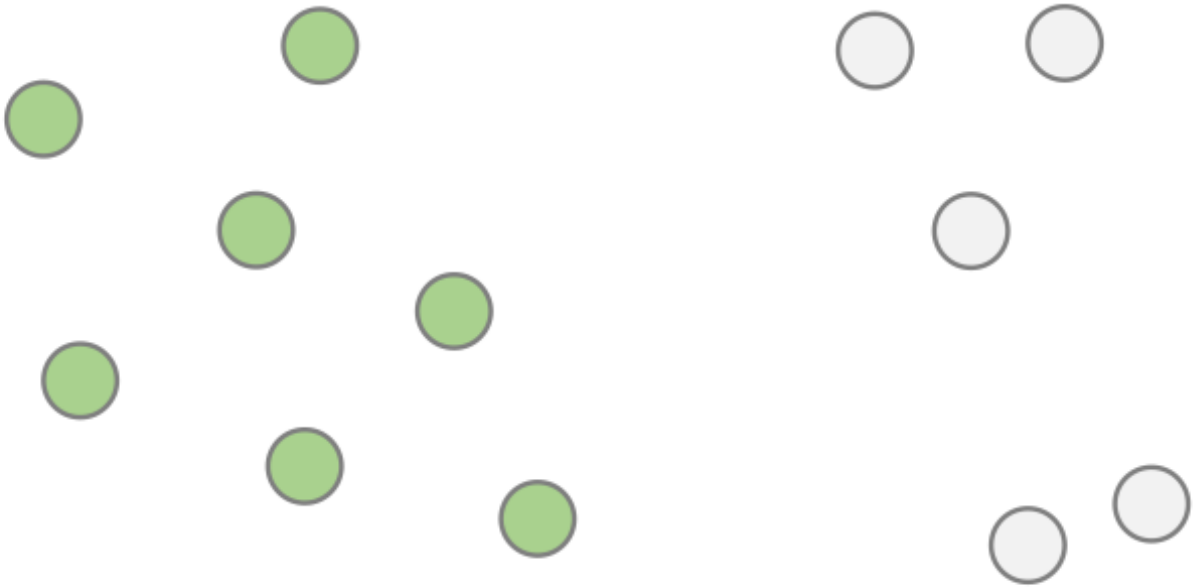
Example



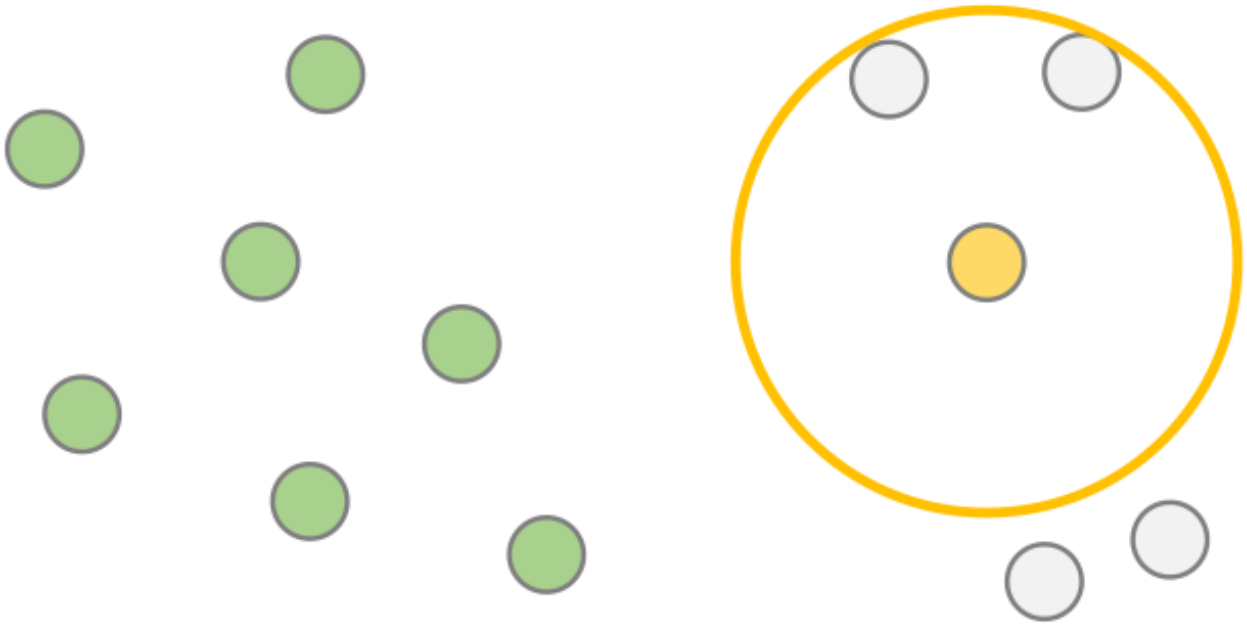
Example



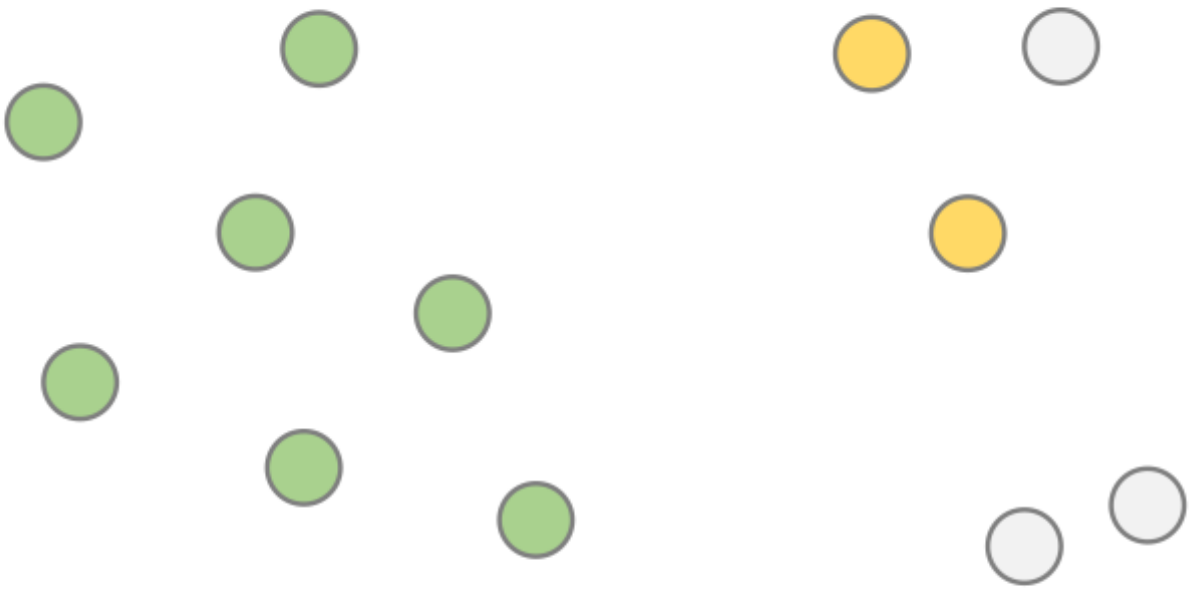
Example



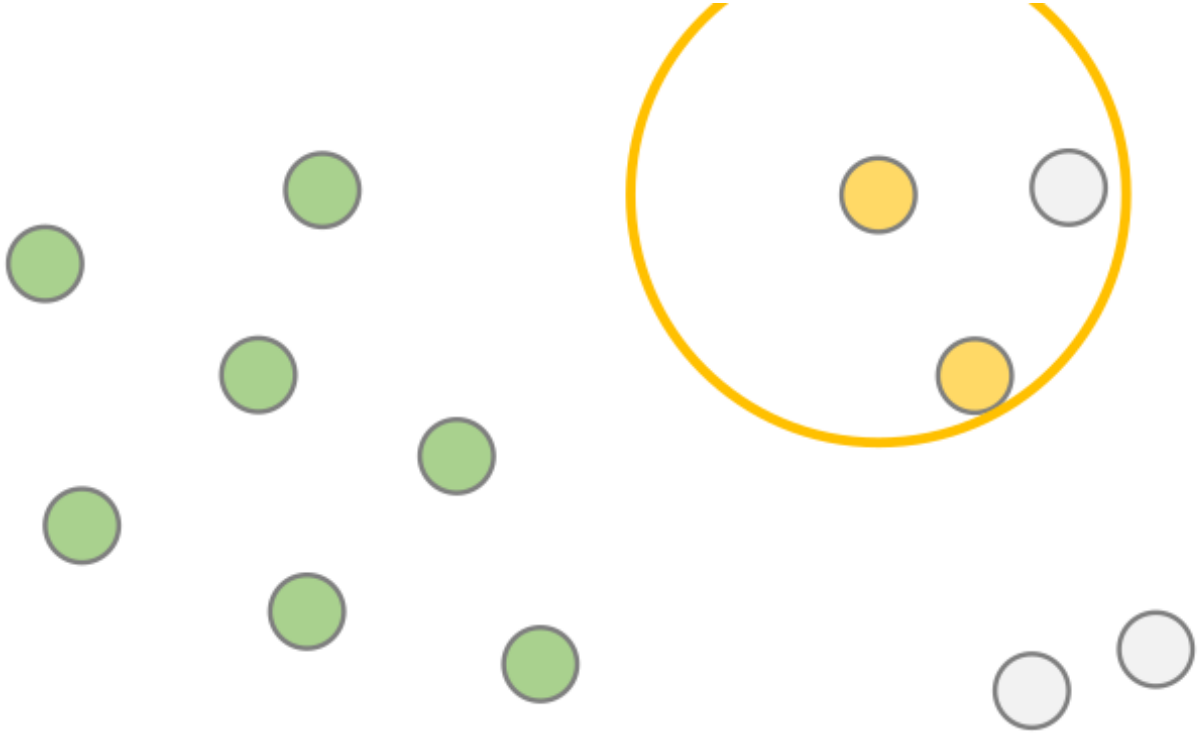
Example



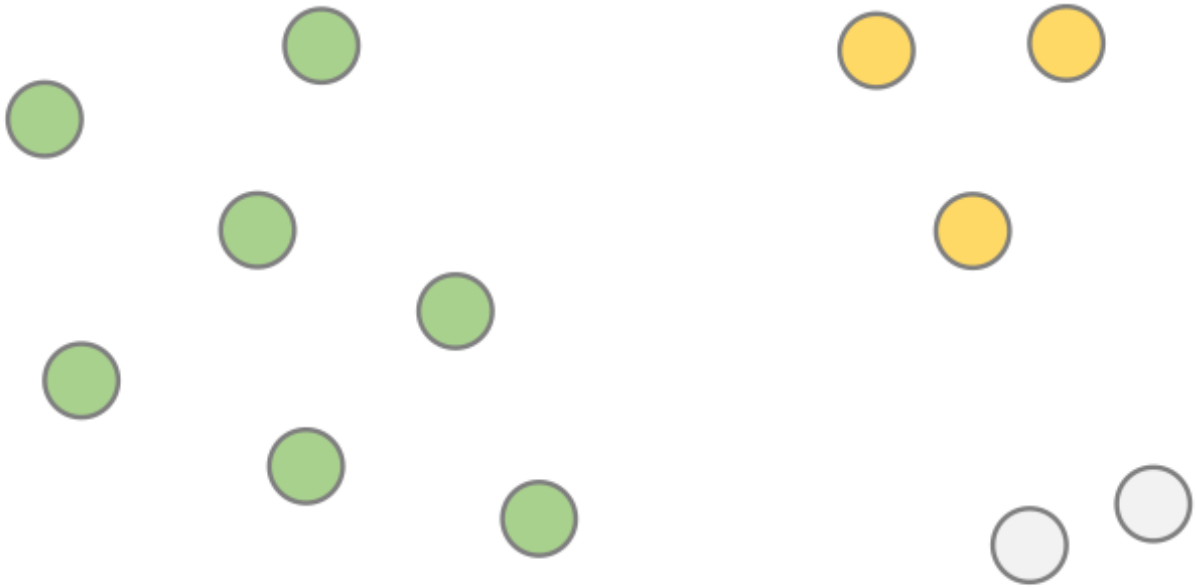
Example



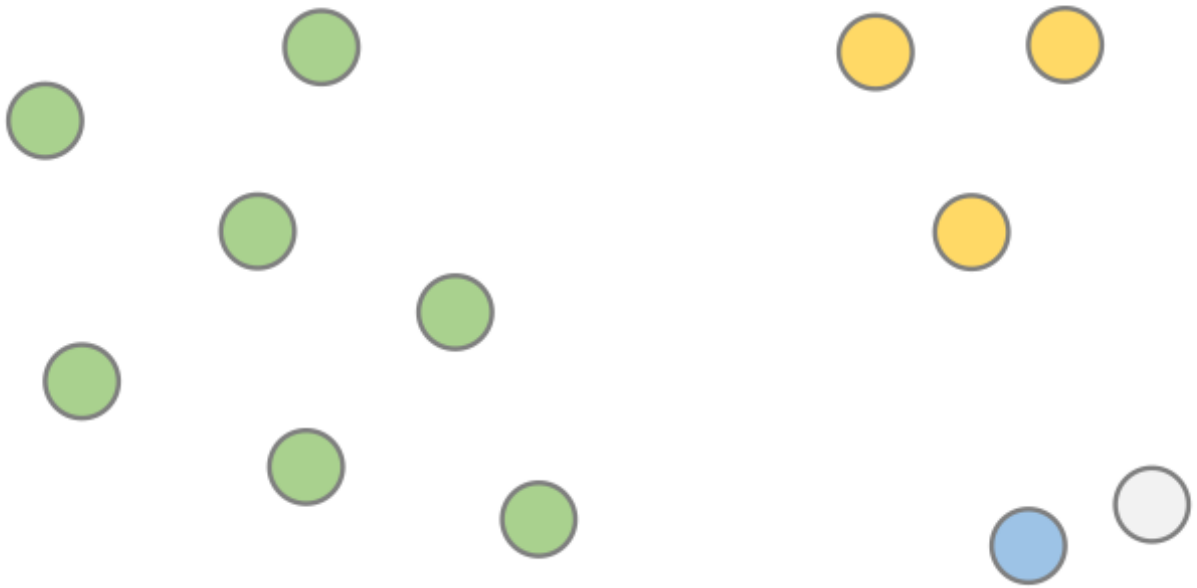
Example



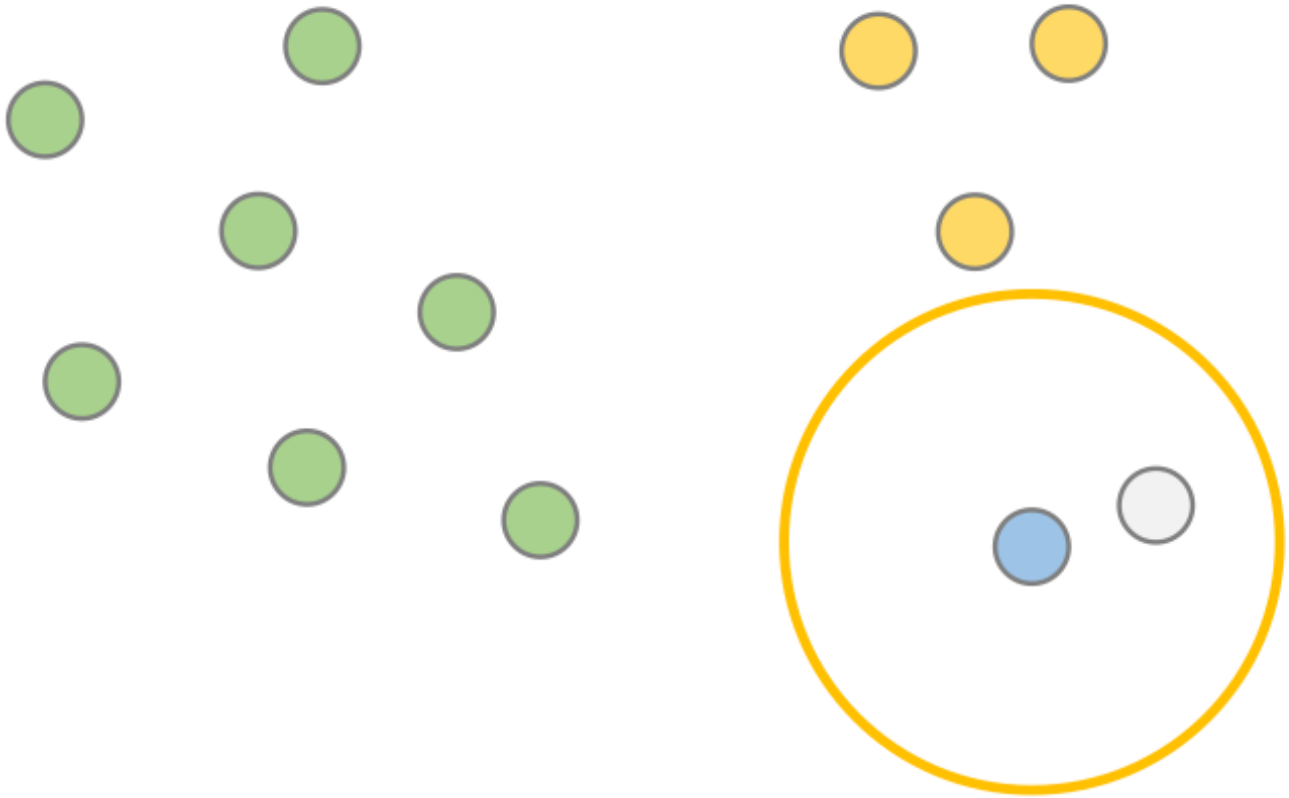
Example



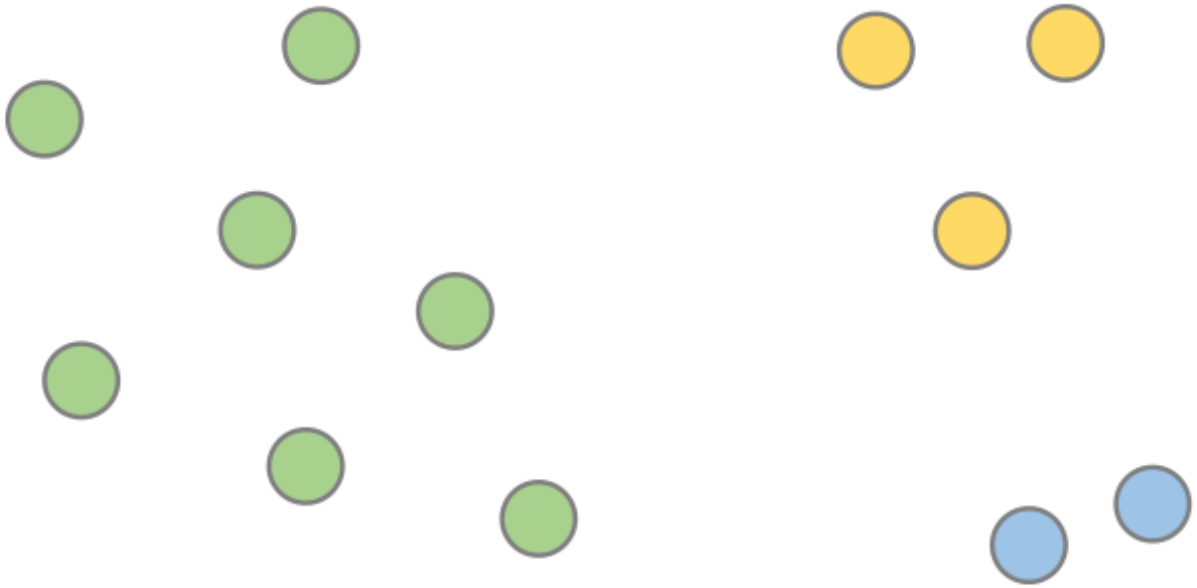
Example



Example



Example



Pros and contras

Advantages	**Disadvantages**
- can find **nonlinearly** separable clusters	- not deterministic
- we don't have to specify the number of cluster	- hard to set ε and K
- find the outliers	- distance metric
- running time $O(N \log N)$	

Hierarchical clustering

- the main disadvantage of k-means clustering is that we have to specify the k number of clusters
- **hierarchical clustering** does not need this parameter
- we build a tree-like structure that essentially contains all the possible k number cluster
- **Scaling** is very important (z-transormation)

Hierarchical clustering

About the algorithm

- it seeks to build a hirearchy of clusters
- **agglomerative approach**
- each item starts in its own cluster and pairs of clusters are merged as one moves up the hierarchy
- the merges and splits are determined in a greedy manner
- it has $O(N^3)$ running time complexity
- the results of hierarchical clustering is usually presented in a so-called **dendogram**

Hierarchical clustering

The Algorithm

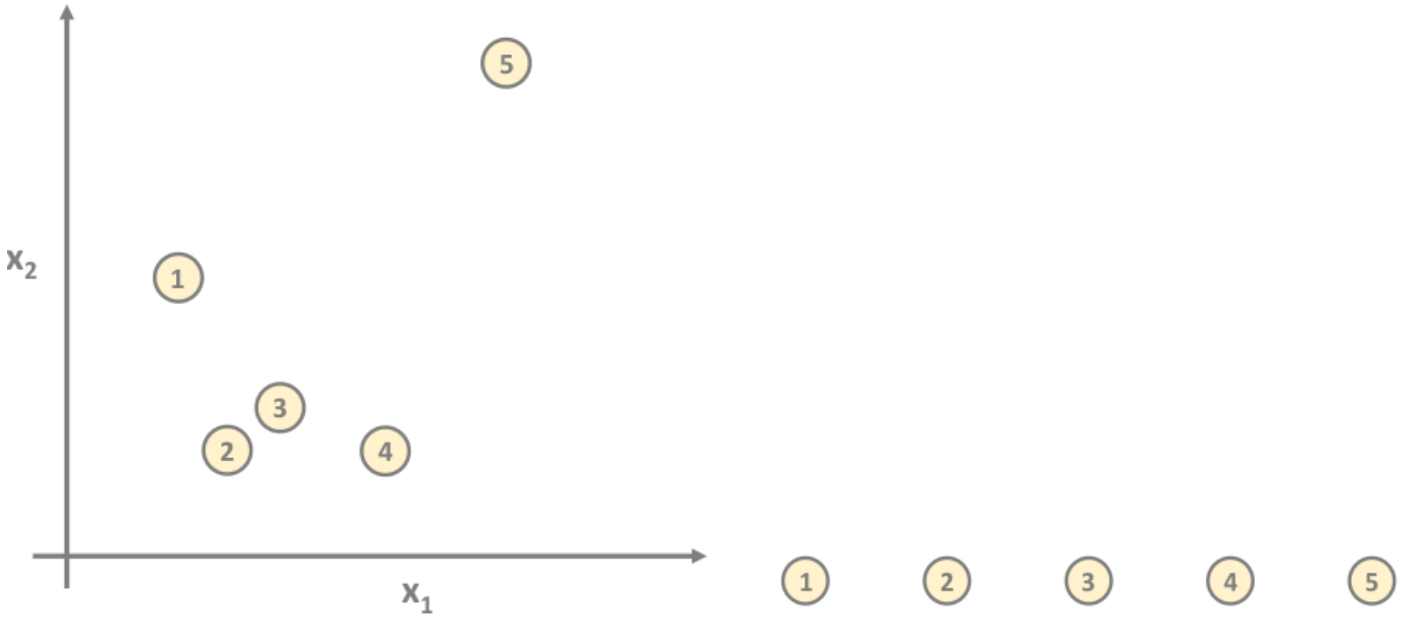
- initialize each data point as a cluster
- find the two closest clusters and **merge them**, the **dendogram** represents these merge operations
- repeat the algorithm until just a single cluster left

Hierarchical clustering

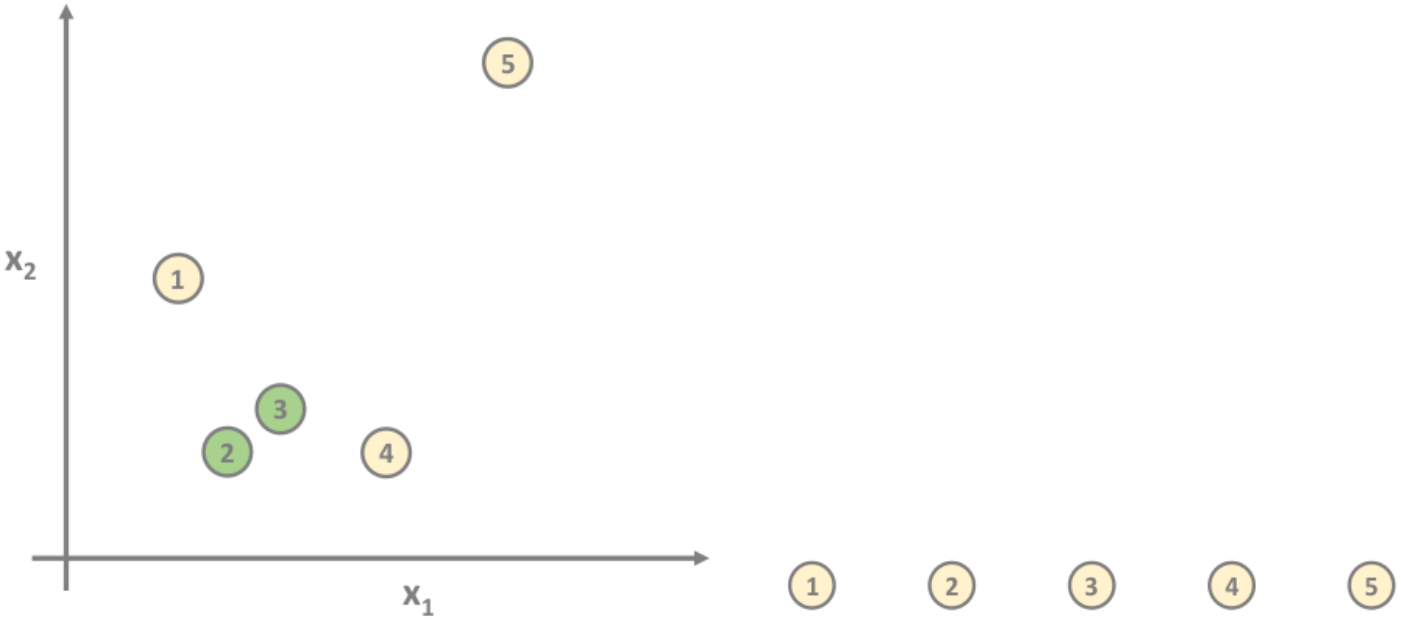
How to measure the distance between clusters?

- average distance between cluster elements
- minimal distance between cluster elements
- distance between the centers of the clusters
- distance metrics

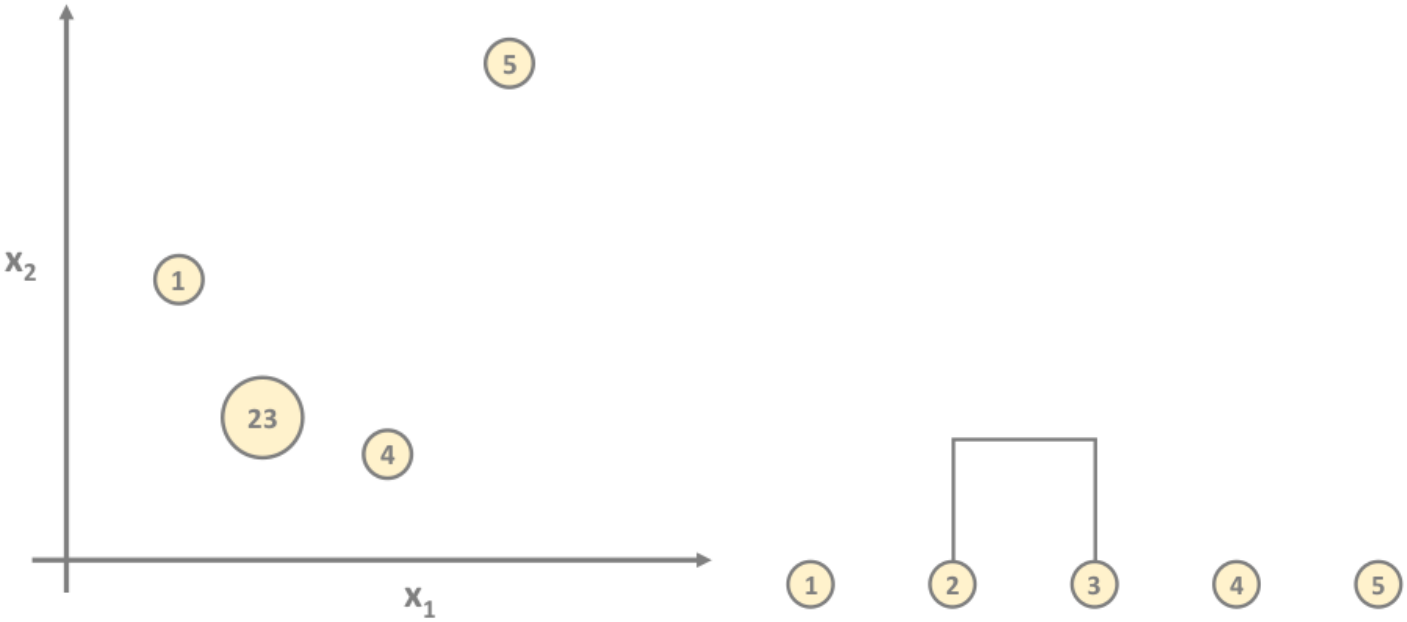
Example



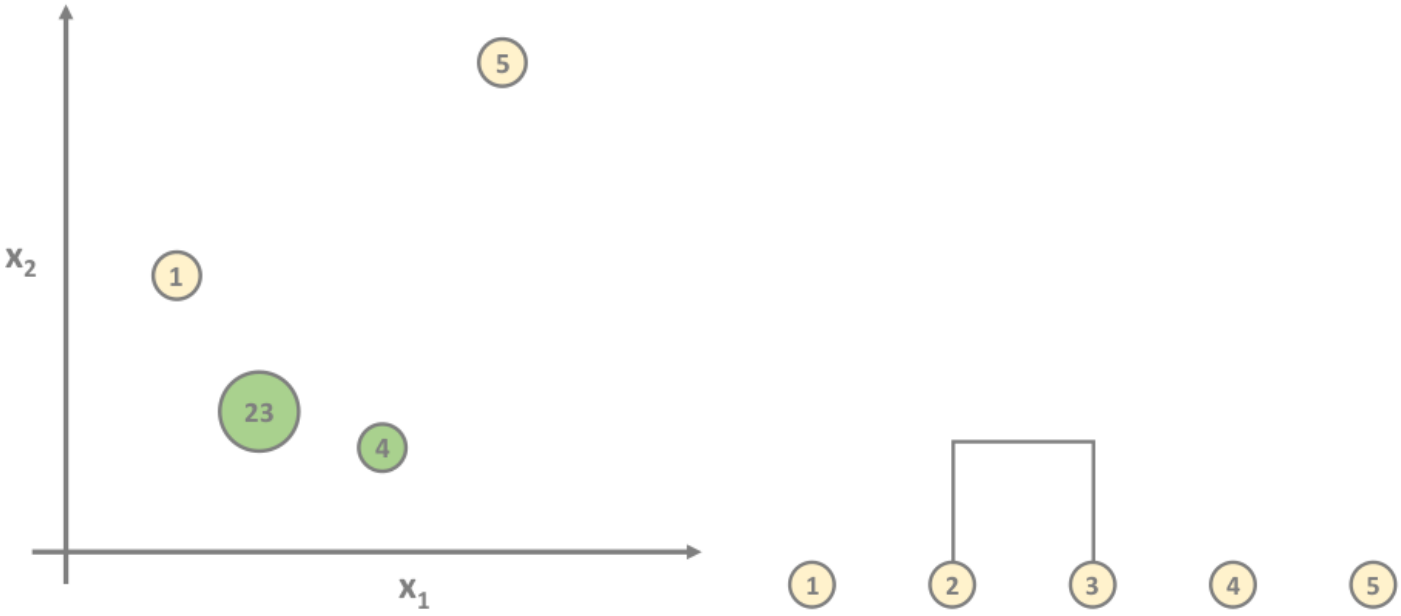
Example



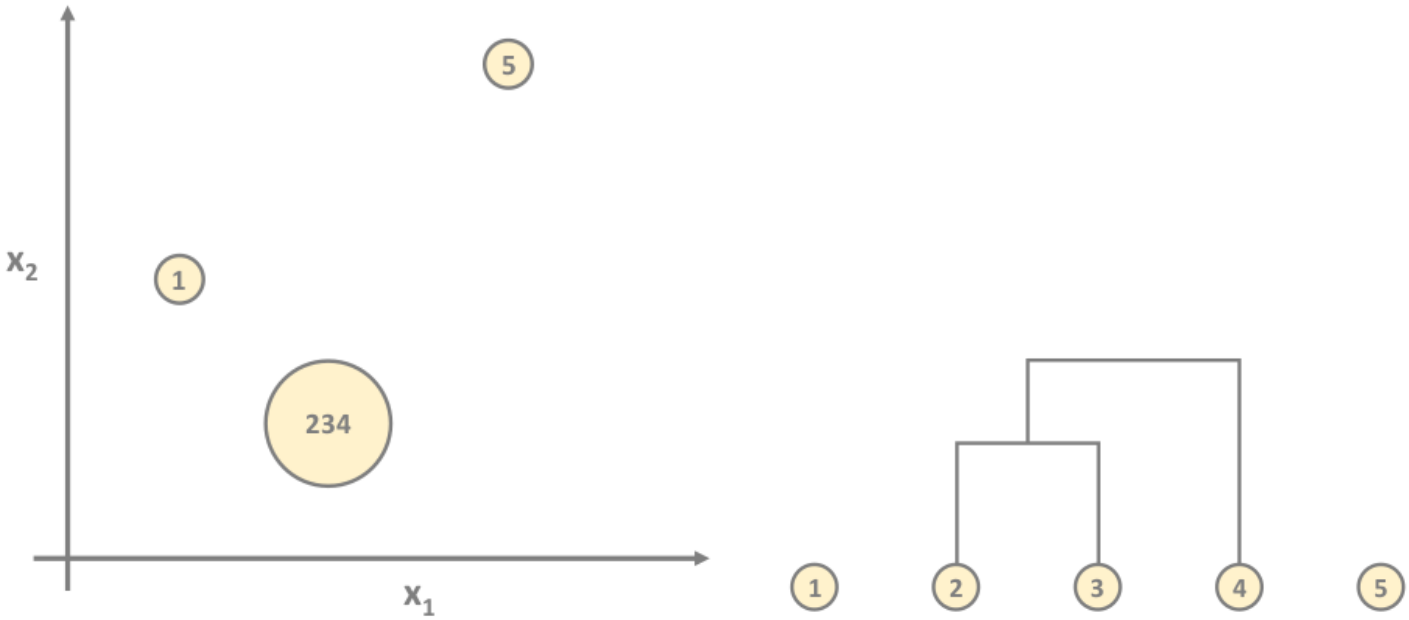
Example



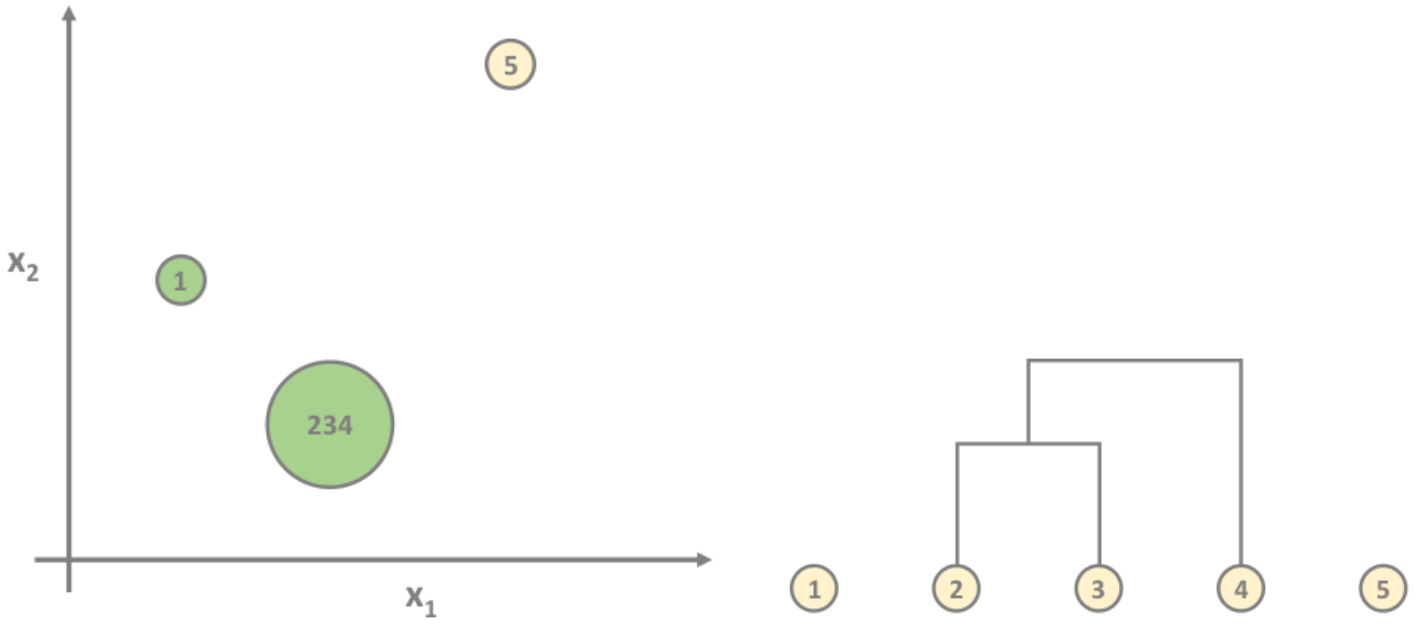
Example



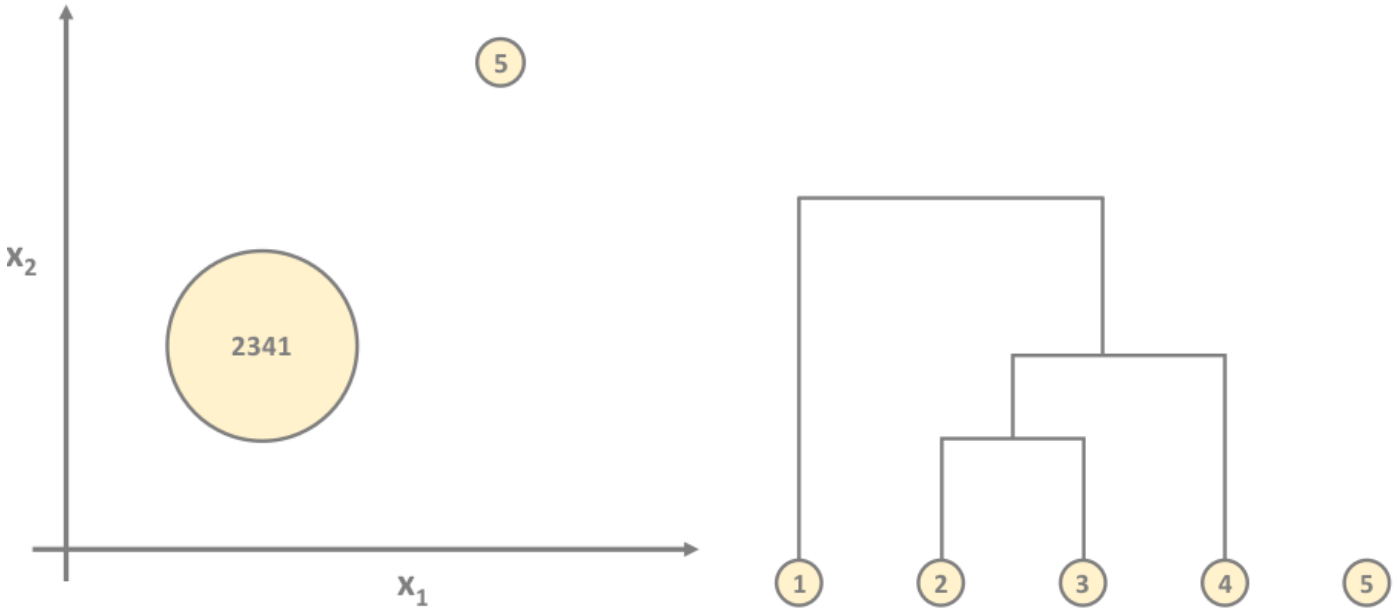
Example



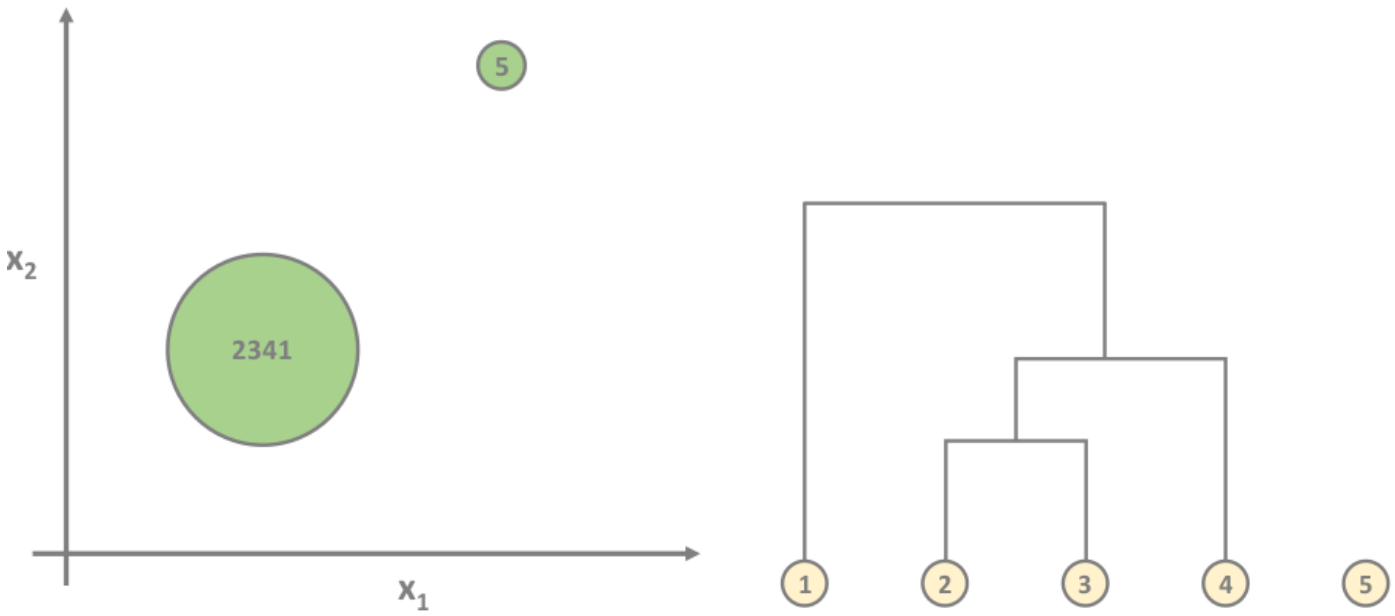
Example



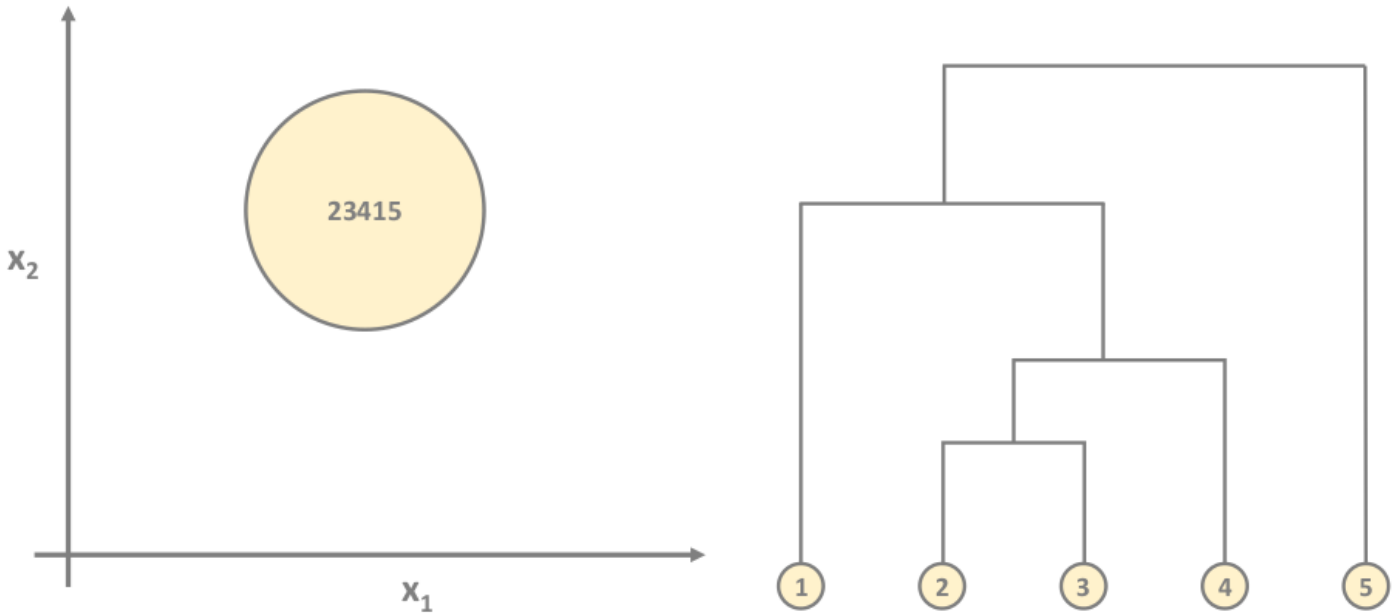
Example



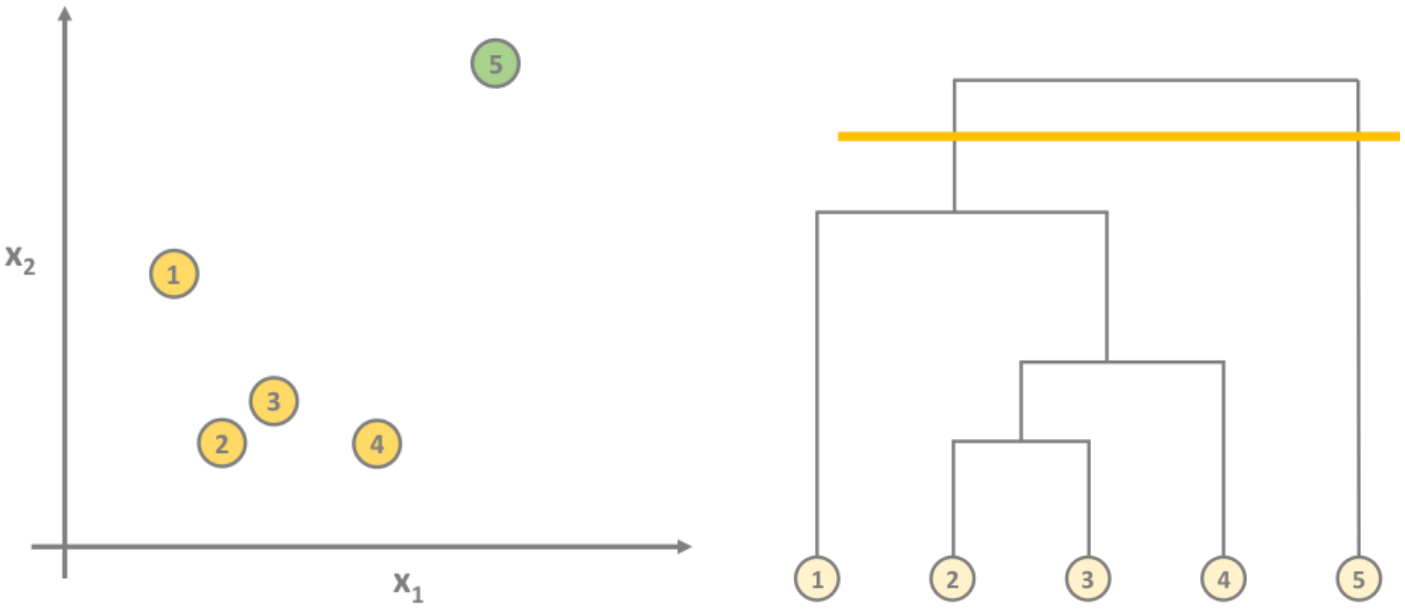
Example



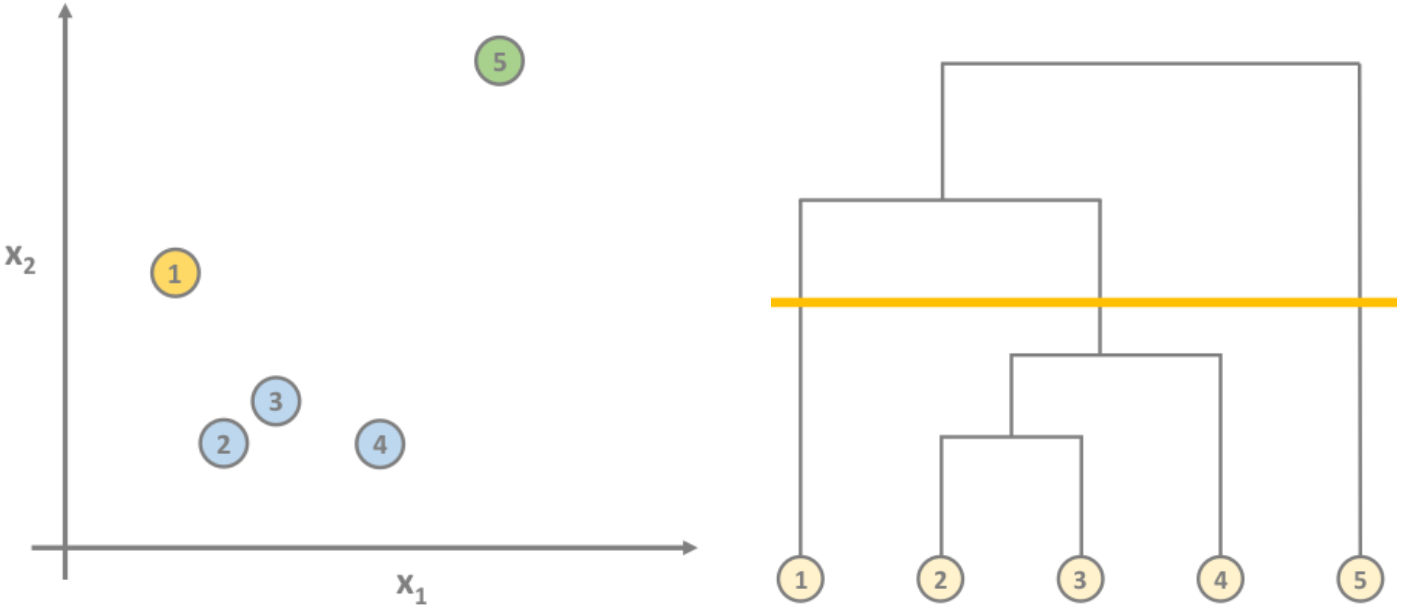
Example



Example

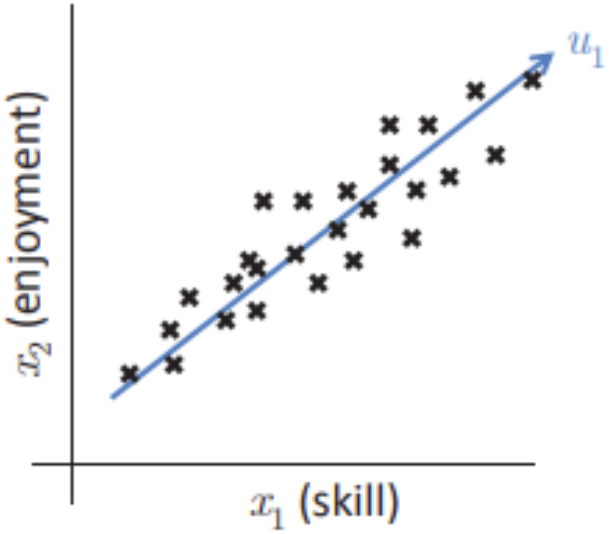


Example



Principal Component Analysis

Example: survey of pilots for RC helicopters



- ${x_1}^{(i)}$: skill of pilot i
- ${x_2}^{(i)}$: enjoyment of pilot i

$\Rightarrow$ strongly correlated

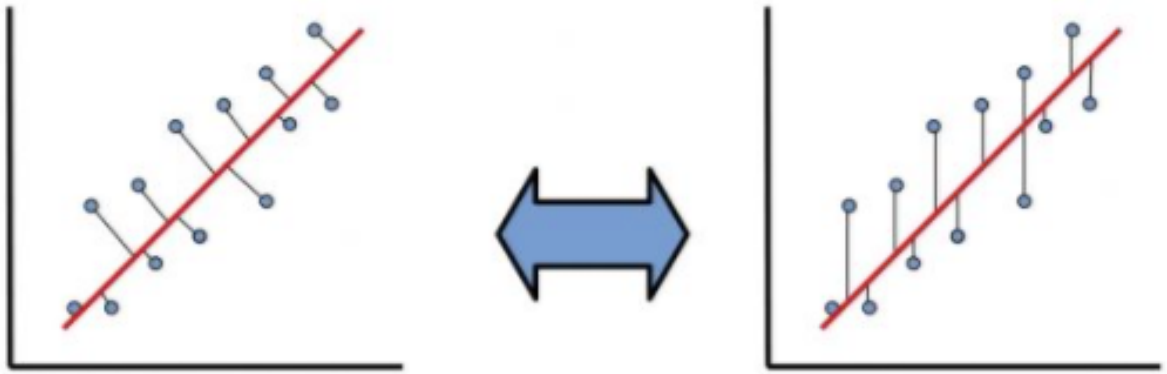
- Data actually lies along some diagonal axis u_1 (**eigenvector**) that computes pilot "aptitude", with small amount of noise lying off-axis

Aim

- identify this subspace using eigenvectors

Difference to linear regression

We don't want to use **Linear Regression**!



The difference

PCA wants to minimizes the geometric distances between the data points and the line (hyperplane).

$\Rightarrow$ less features, reduced overfit

The algorithm

Data

- We have n pieces of data rows: $x^{(i)} \in \mathbb{R}^d$
- The aim is to project the data into $\mathbb{R}^k$ by losing the least information as possible ($k < d$).
  $\Rightarrow$ Dimensionality reduction = Compression
- Pre-process the data by **standardizing** all features!

- compute the covariance matrix $\Sigma = \frac{1}{n} \sum_{i=1}^{n} x^{(i)} \left(x^{(i)}\right)^T$
- Find eigenvalues ($D \in \mathbb{R}^{d \times d}$) and eigenvectors ($V \in \mathbb{R}^{d \times d}$) of $\Sigma \in \mathbb{R}^{d \times d}$
- Retain the k eigenvectors with largest eigenvalues
- Projection: $\hat{V} \in \mathbb{R}^{d \times k}$ contains these eigenvectors $\Rightarrow x^{(i)} \hat{V} \in \mathbb{R}^k$

The result will be a new dataset: $\hat{x}^{(i)} \in \mathbb{R}^k$ with less feature than originially

How to choose k?

To choose k note that

- average squared projected error (ASPE) $= \frac{1}{n}\sum_{i=1}^{n}\|x^{(i)} - \hat{x}^{(i)}\|^2$
- total variation in the data (TV) $= \frac{1}{n}\sum_{i=1}^{n}\|x^{(i)}\|^2$
- Typically choose k to be the smallest value so that the error is upper bounded.

  To bound the error at 1%, choose k the following way: $\frac{ASPE}{TV} \leq 0.01$

  This is equivalent to retaining 99% of the variance.

Trick

$$\frac{ASPE}{TV} = \frac{\text{sum of the k largest eigenvalues of } \Sigma}{\text{sum of the eigenvalues of } \Sigma}$$